American International University-Bangladesh (AIUB)
Faculty of Science and Technology

A Two-Stage Hierarchical Multiclass Network Intrusion Detection System with Tempered Class Weighting and a Reject Option for Imbalanced Network Flows

Abidur Rahman (22-47460-2)
Parthib Datta (22-47465-2)
Md. Ebnul Ahsan (22-47464-2)
Mhamudul Hasan Ovi (22-47381-2)

A Thesis submitted for the degree of Bachelor of Science (BSc)in
Computer Science and Engineering (CSE) at
American International University Bangladesh (AIUB)
Faculty of Science and Technology (FST)

Summer 2025-2026 Semester
Submission Date: Month, Year

Declaration

This thesis is composed of our original work, and contains no material previously published or writtenby another person except where due reference has been made in the text. We have clearly stated the contribution of others to our thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financialand any other original research work used or reported in our thesis. The content of our thesisis the result of work we have carried out since the commencement of the Thesis.

We acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate we have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors forany jointly authored works included in the thesis.

Abidur Rahman
22-47460-2
Department of Computer Science

Parthib Datta
22-47465-2
Department of Computer Science

Md. Ebnul Ahsan
22-47464-2
Department of Computer Science

Mhamudul Hasan Ovi
22-47381-2
Department of Computer Science

Approval

The thesis titled **"A Two-Stage Hierarchical Multiclass Network Intrusion Detection System with Tempered Class Weighting and a Reject Option for Imbalanced Network Flows"** has been submitted to the following respected members ofthe board of examiners of the department of computer science in partial fulfilment of the requirementsfor the degree of Bachelor of Science in Computer Science on **(date of defense)** and has been acceptedas satisfactory.

DR. ABHIJIT BHOWMIK
Associate Professor & Supervisor
Department of Computer Science
American International University-Bangladesh

S M ABDULLAH SHAFI
Lecturer & External
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Muhammad Firoz Mridha
Professor & Head (UG)
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Khandaker Tabin Hasan
Professor & Associate Dean
Faculty of Science and Technology
American International University-Bangladesh

Prof. Dr. Dip Nandi
Professor & Dean
Faculty of Science and Technology
American International University-Bangladesh

Acknowledgement

We are deeply grateful to our supervisor for the guidance, patience and critical feedback that shaped this work from the first experiments to the final report. We also thank the faculty of the Department of Computer Science, American International University-Bangladesh, for the knowledge and resources that made this thesis possible.

We acknowledge the Canadian Institute for Cybersecurity and the UNSW Canberra Cyber Range Lab for making the CIC-UNSW-NB15, UNSW-NB15 and related datasets publicly available, and the developers of the open-source Python libraries on which every experiment in this thesis depends.

Finally, we thank our families and friends for their constant encouragement and support throughout our undergraduate studies.

Table of Content

DECLARATION II
APPROVAL III
ACKNOWLEDGEMENT IV
TABLE OF CONTENT V
LIST OF FIGURES VII
LIST OF TABLES VIII
LIST OF ABBREVIATIONS IX
ABSTRACT XI
KEYWORDS XII
CHAPTER 1 13
INTRODUCTION 13
1.1 INTRODUCTION 13
1.2 PROBLEM STATEMENT 13
1.3 PROBLEM BACKGROUND 14
1.4 RESEARCH OBJECTIVES 14
1.5 RESEARCH QUESTIONS 15
1.6 MOTIVATIONS 15
1.7 FLOW OF RESEARCH 16
1.8 SIGNIFICANCE OF THE RESEARCH 16
1.9 RESEARCH CONTRIBUTION 17
1.10 THESIS ORGANIZATION 17
1.11 SUMMARY 18
LITERATURE REVIEW 19
2.1 INTRODUCTION 19
2.2 LITERATURE REVIEW 19
2.3 PROBLEM ANALYSIS 27
2.4 SUMMARY 29
CHAPTER 3 30
PROPOSED MODEL 30
3.1 INTRODUCTION 30
3.2 FEASIBILITY ANALYSIS 30
3.3 REQUIREMENT ANALYSIS 31
3.4 RESEARCH METHODOLOGY 33
3.5 SYSTEM DESIGN AND ARCHITECTURE 34

3.6 IMPLEMENTATION AND SIMULATION 38
3.7 TOOLS AND TECHNOLOGIES USED 46
3.8 SUMMARY 47
CHAPTER 4 48
IMPLEMENTATION AND TESTING 48
4.1 INTRODUCTION 48
4.2 SYSTEM SETUP AND ENVIRONMENT 48
4.3 IMPLEMENTATION OF THE SYSTEM 50
4.4 EVALUATION AND TESTING 52
4.5 RESULTS AND DISCUSSION 54
4.6 SUMMARY 69
CHAPTER 5 70
STANDARDS, CONSTRAINTS AND MILESTONES 70
5.1 STANDARDS AND COMPLIANCE 70
5.2 SUSTAINABILITY CONSIDERATIONS 71
5.3 SOCIETAL IMPACT 72
5.4 ETHICAL CONSIDERATIONS 72
5.5 SECURITY AND PRIVACY CONSIDERATIONS 73
5.6 DESIGN CONSTRAINTS AND COMPONENT CONSTRAINTS 73
5.7 DATA CONSTRAINTS AND COMPUTATIONAL CONSTRAINTS 74
5.8 BUDGET CONSTRAINTS 75
5.9 TASK BREAKDOWN AND PROJECT TIMELINE 75
5.10 MILESTONES AND DELIVERABLES 76
5.11 GANTT CHART 76
5.12 SUMMARY 77
CHAPTER 6 79
CONCLUSION 79
6.1 SUMMARY OF THE STUDY / WORK DONE 79
6.2 KEY CONTRIBUTIONS 80
6.3 LIMITATIONS OF THE STUDY 80
6.4 FUTURE WORK 81
REFERENCES 82
APPENDIX 87

List of Figures

FIGURE 1.1 FLOW OF THE RESEARCH, FROM DATA COLLECTION TO RESULT ANALYSIS 16

FIGURE 3.1 ARCHITECTURE OF THE PROPOSED TWO-STAGE INTRUSION DETECTION SYSTEM 35

FIGURE 3.2 CLASS DISTRIBUTION OF THE CURATED FILE ON A LOGARITHMIC SCALE 39

FIGURE 3.3 EXPERIMENTAL PROTOCOL USED FOR EVERY EXPERIMENT (FLOW COUNTS FOR THE CURATED FILE) 45

FIGURE 4.1 MACRO-F1 PROGRESSION ON THE CURATED TEST SET, WITH 95% BOOTSTRAP CONFIDENCE INTERVALS WHERE THEY WERE COMPUTED 59

FIGURE 4.2 CONFUSION MATRIX OF THE FINAL SYSTEM, FULL-RELEASE TEST SET, 521,361 FLOWS 61

FIGURE 4.3 ROBUSTNESS OF DETECTION, PANEL A, AND ATTRIBUTION, PANEL B, AS THE PERTURBATION BUDGET GROWS 65

FIGURE 5.1 PROJECT GANTT CHART (INDICATIVE WEEKS) 77

List of Tables

TABLE 3.1 FUNCTIONAL REQUIREMENTS 31
TABLE 3.2 NON-FUNCTIONAL REQUIREMENTS 32
TABLE 3.3 CLASS DISTRIBUTION OF THE CURATED CIC-UNSW-NB15 FILE (BEFORE DEDUPLICATION) 38
TABLE 3.4 PREPROCESSING STEPS IN EXECUTION ORDER (CURATED FILE) 40
TABLE 3.5 ENGINEERED BEHAVIOURAL FEATURES 41
TABLE 3.6 SERVICE-IDENTITY FEATURES 42
TABLE 3.7 STAGE-B ENSEMBLE MEMBERS 42
TABLE 3.8 MODEL SETTINGS 43
TABLE 3.9 DATA PARTITIONS 44
TABLE 3.10 TOOLS AND TECHNOLOGIES 46
TABLE 4.1 HARDWARE SPECIFICATION 48
TABLE 4.2 BASELINE CLASSIFIERS WITH AND WITHOUT SUPERVISED FEATURE SELECTION, CURATED TEST SET, 61,235 FLOWS 55
TABLE 4.3 IMBALANCE TREATMENTS COMPARED ON THE TEST SET, CURATED TEST SET, 61,235 FLOWS 56
TABLE 4.4 ENSEMBLE DESIGNS ON THE 44 BASE FEATURES, CURATED TEST SET, 61,235 FLOWS 57
TABLE 4.5 PER-CLASS F1 CHANGE FROM SERVICE-IDENTITY FEATURES, SAME-RUN PAIRED COMPARISON 58
TABLE 4.6 TOP 15 STAGE-B FEATURES BY IMPORTANCE WITH SERVICE IDENTITY 58
TABLE 4.7 FULL-RELEASE CLASS DISTRIBUTION AFTER DEDUPLICATION 59
TABLE 4.8 FINAL SYSTEM, OVERALL TEST METRICS, FULL-RELEASE TEST SET, 521,361 FLOWS 60
TABLE 4.9 FINAL SYSTEM, PER-CLASS RESULTS, FULL-RELEASE TEST SET, 521,361 FLOWS 60
TABLE 4.10 REJECT-CLASS ACCOUNTING FOR THE FINAL SYSTEM, FULL-RELEASE TEST SET, 521,361 FLOWS 61
TABLE 4.11 IMPROVEMENT LADDER ON THE FULL RELEASE WITH PAIRED SIGNIFICANCE, ALL CONFIGURATIONS FROM ONE RUN, FULL-RELEASE TEST SET, 521,361 FLOWS 62
TABLE 4.12 PAIRED SIGNIFICANCE OF EACH COMPONENT, 2,000 BOOTSTRAP RESAMPLES 62
TABLE 4.13 APPROACHES THAT DID NOT IMPROVE THE SYSTEM 63
TABLE 4.14 THE TWELVE MOST CONFUSED FAMILY PAIRS, COMPARING CLASS OVERLAP WITH EXACT LABEL AMBIGUITY 64
TABLE 4.15 ROBUSTNESS OF THE DEPLOYED SYSTEM UNDER RANDOM NOISE AND TARGETED EVASION, FULL-RELEASE TEST SET, 521,361 FLOWS 65
TABLE 4.16 OUTCOME RATES UNDER THE EVASION AND MISATTRIBUTION OBJECTIVES, FULL-RELEASE TEST SET, 521,361 FLOWS 66
TABLE 4.17 EXTERNAL CHECKS ON OTHER DATASETS 66
TABLE 4.18 THIS WORK ALONGSIDE THE REVIEWED STUDIES 67
TABLE 5.1 STANDARDS AND FRAMEWORKS RELEVANT TO THIS WORK 71
TABLE 5.2 TASK BREAKDOWN 75
TABLE 5.3 MILESTONES AND DELIVERABLES 76
TABLE A.1 FORMULAS OF THE 31 ENGINEERED BEHAVIOURAL FEATURES 87
TABLE A.2 SOURCE OF EACH MAIN RESULT 88

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AIUB	American International University-Bangladesh
CI	Confidence Interval
CIC	Canadian Institute for Cybersecurity
CPU	Central Processing Unit
CSV	Comma-Separated Values
DES	Dynamic Ensemble Selection
DoS	Denial of Service
DT	Decision Tree
ET	Extra-Trees (Extremely Randomised Trees)
FN	False Negative
FP	False Positive
FPR	False Positive Rate
FR	Functional Requirement
GPU	Graphics Processing Unit
IAT	Inter-Arrival Time
IDS	Intrusion Detection System
IP	Internet Protocol
IQR	Interquartile Range
ISO	International Organization for Standardization
IEC	International Electrotechnical Commission
k-NN	k-Nearest Neighbours
ML	Machine Learning
NFR	Non-Functional Requirement
NIDS	Network Intrusion Detection System
NIST	National Institute of Standards and Technology
OLA	Overall Local Accuracy
OOF	Out-of-Fold
RAM	Random Access Memory
RF	Random Forest
ROC-AUC	Area Under the Receiver Operating Characteristic Curve

Abbreviation	Meaning
SELV	Selection-Validation split
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SOC	Security Operations Centre
TCP	Transmission Control Protocol
TTL	Time To Live
UDP	User Datagram Protocol
XAI	Explainable Artificial Intelligence
XGB	XGBoost (eXtreme Gradient Boosting)
α	Class-weight tempering exponent
W	Per-class combiner weight matrix

Abstract

Most of the Machine-learning based network intrusion detection systems (NIDS) scores near perfect accuracy on public benchmarks, but the same systems fail to recognize the rare attack families that matter most in practice. Two reasons recur in the literature: severe class imbalance, which accuracy hides, and evaluation protocols that let duplicated or label-leaking records cross the train–test boundary. This thesis designs, builds and rigorously evaluates a multi-class NIDS that addresses both problems on CIC-UNSW-NB15, a 2024 re-extraction of the UNSW-NB15 traffic with CICFlowMeter flow features covering benign traffic and nine attack families.

The proposed system has two stages. At the first stage a binary gate separates the benign flows from the malicious flows and then a ensemble of seven tree-based classifier identifies the attacks and assigns them to its family. The ensemble members deliberately differ in how strongly they compensate for class imbalance, through a tempered class-weighting exponent α, and their outputs are fused by a learned per-class weight matrix optimised directly for end-to-end macro-F1. At the realistic base rate, the second stage also learns a reject output for benign flows that slip past the gate. The pipeline is leakage-safe by construction: duplicates are removed before splitting, every label-dependent step is fitted on training data only, combiner weights are fitted on cross-fitted out-of-fold predictions, and the test set is scored once. Every claimed improvement is tested with a paired bootstrap over 2,000 resamples.

On the curated dataset, macro-F1 rises from 0.534 for a decision tree and 0.581 for XGBoost to 0.649 [0.631, 0.666] with tempered weighting, the two-stage hierarchy, the ensemble and 31 engineered behavioural features. The score went up by 0.098, to 0.746. We're 95% sure the real score is somewhere between 0.727 and 0.763. Then we gave the final system a much harder test: the full dataset, with 3.5 million pieces of traffic. In this bigger set, 97.5% of the traffic is harmless, so attacks are much rarer, just like on a real network. Here the system scored 0.687, and we're 95% sure the real score is between 0.671 and 0.704. It labeled 98.7% of all the traffic correctly. The reject class cuts benign false alarms fourfold. Tempered weighting (+0.039) contributes more than the whole ensemble (+0.018). Hyperparameter tuning, dynamic ensemble selection and threshold tuning give no established improvement.

Further analysis shows that the remaining errors arise from class overlap in the flow-feature

representation rather than from label noise or model capacity. A robustness study under physically consistent perturbations finds the system brittle to small measurement noise. A random-forest gate preserves detection far better than XGBoost at a small clean cost, and a previously unexamined attacker objective, misattribution, can corrupt family labels while alerts keep firing. The results show that careful imbalance handling, richer flow information and honest evaluation matter more than model complexity for practical intrusion detection.

Keywords

network intrusion detection, class imbalance, ensemble learning, xgboost, cic-unsw-nb15, hierarchical classification, data leakage, adversarial robustness, macro-f1

Chapter 1

Introduction

1.1 Introduction

Modern organizations use interconnected systems. Attack surface can be narrowed down by using firewall and access control, but they often fail to distinguish between legitimate use and exploitation, scanning or denial of service.

Network intrusion detection systems (NIDS) try to fill this gap. They watch traffic and raise an alert when behaviour breaks the expected security policy (Scarfone & Mell, 2007). Systems based on signatures perform well in this regard for known assaults. A distinct strategy is used by machine-learning systems, which learn from flow statistics. how harmful and legitimate traffic seem, allowing them to potentially identify assaults they have never seen (Ahmad et al., 2021).

High overall accuracy is not enough, however. A useful detector must keep false alarms low. It must also name the type of attack correctly and still work for rare attack types, and it must stay reliable when the input data is noisy or deliberately manipulated (Apruzzese et al., 2022; Arp et al., 2022).

This thesis studies that harder problem on CIC-UNSW-NB15 (Mohammadian et al., 2024) which is a 2024 version of the UNSW-NB15 traffic (Moustafa & Slay, 2015) with the CICFlowMeter flow extractor (Lashkari et al., 2017). The data is extremely imbalanced where 97.5% of flows are benign, and the rarest attack family has only a few hundred examples. A model can therefore look accurate while missing exactly the attacks that matter. For this reason, the thesis uses macro-F1, not accuracy, as its primary measure of success.

1.2 Problem Statement

Published ML-based network intrusion detectors frequently report accuracies above 99%. In multi-class detection, however, such figures conceal poor performance on rare attack families, because benign traffic and a few common attacks dominate the data. T Additionally, they are frequently exaggerated by evaluation procedures that allow identical or almost identical records to show up in both the test and training data. Therefore, it is still

unknown how well a flow-based detector can detect every attack family when imbalance is appropriately managed and assessment is leakage-free, as well as whether design decisions truly enhance this capability

1.3 Problem Background

Publicly accessible benchmark datasets such as KDD Cup 99, NSL-KDD, UNSW-NB15, and CIC-IDS2017 are used in the majority of machine-learning NIDS research (Moustafa & Slay, 2015; Sharafaldin et al., 2018). These datasets share two properties that complicate evaluation. First, they are highly imbalanced: in the dataset used here, 80% of records are benign while the rarest attack family, Worms, makes up only 0.05%. A classifier can therefore score high accuracy while missing rare attacks entirely. Common remedies such as synthetic oversampling with SMOTE (Chawla et al., 2002) are often applied without checking whether they help. Second, these datasets contain many duplicate records (about 32% here). It has been demonstrated that duplicate records in KDD Cup 99 can skew learning and assessment (Tavallaee et al., 2009), and data leaking in general is a known cause of overly optimistic security ML findings (Arp et al., 2022). Moreover, a lot of research report accuracy without providing per-class findings or confidence ranges, evaluate on a single dataset, and withhold the code required to verify their procedure. When combined, these methods make it difficult to determine which reported advances are genuine.

1.4 Research Objectives

The main objective of this research is to design, implement and rigorously evaluate a leakage-safe, imbalance-aware machine-learning system that detects network intrusions and identifies their attack family from flow records. The specific objectives are:

- to build a preprocessing and evaluation pipeline that removes duplicate records and fits every label-dependent step on training data only, so that reported scores are free of leakage;
- to measure the effect of different class-imbalance treatments (class weighting, tempered weighting and SMOTE oversampling) on rare-family detection, using macro-F1 as the primary metric;
- to design a two-stage architecture, a detection gate followed by an attack-family

ensemble, and to quantify each component's contribution with paired statistical tests;

- to investigate whether additional flow information (engineered behavioural features and service identity) improves family identification more than additional model complexity;
- to evaluate the final system at the realistic benign base rate of the full dataset release and under perturbed inputs, to assess its practical reliability.

1.5 Research Questions

The objectives above lead to five research questions:

- How do class-imbalance treatments compare in their effect on the detection of rare attack families, and what degree of rebalancing works best?
- Under what conditions does a two-stage (gate plus family classifier) architecture outperform a single flat multi-class classifier?
- Is performance on this dataset limited mainly by the learning model or by the information contained in the flow representation?
- How does the system perform at the realistic base rate of benign traffic, and how can false alarms from benign flows be reduced?
- How robust are the system's detection and family attribution to small perturbations of the flow features, both random and adversarial?

1.6 Motivations

The motivation for this work comes from the gap between published results and operational needs. It is not sufficient for a security operation system only knows something is wrong. It needs to understand exactly what is wrong to fix it. Different security measurements to be taken for every attack family. The most damaging attack families are also the rarest one. A detector tuned for overall accuracy often misses these attacks. Also, the literature reviewed show a near perfect score but most often does not show per class result, statistical interval or code and for that, practitioners cannot tell for which type of attacks the systems performed the most. A careful, reproducible study on a recent dataset, one that reports what works and what does not, addresses both concerns.

1.7 Flow of Research

The research progressed in six phases. The first phase identified the problem through a review of recent ML-based NIDS literature and an audit of the chosen dataset, which revealed its imbalance and duplication. The second phase built a leakage-safe data preparation pipeline. The third phase established baseline classifiers and then designed and refined the proposed two-stage ensemble through a sequence of controlled experiments, each motivated by the findings of the previous one. The fourth phase evaluated every component with paired statistical tests and extended the evaluation to the full dataset release at its realistic base rate. The fifth phase stress-tested the final system under perturbed inputs. The final phase analyzed the results, including the experiments that failed, and documented them in this report. Figure 1.1 summarizes this sequence.

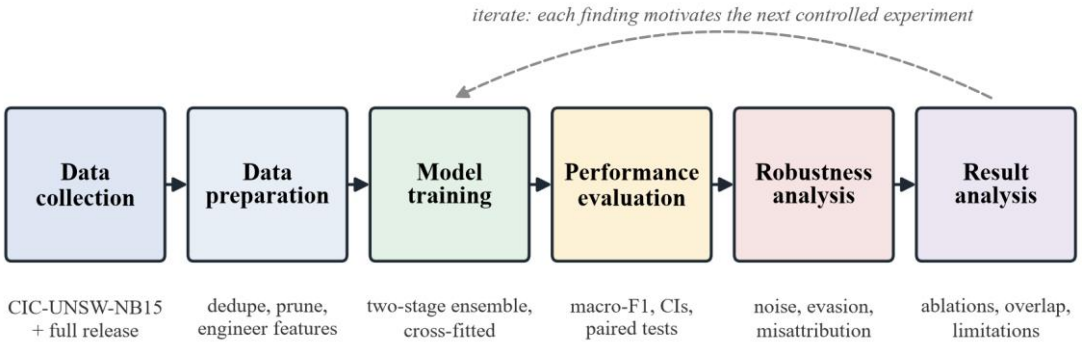


Figure 1.1 Flow of the research, from data collection to result analysis

1.8 Significance of the Research

This research is valuable to three groups. Researchers gain a reproducible evaluation protocol which prevents data leakage. Additionally, a detailed knowledge of which design choices actually improved results on a recent benchmark. It also reports negative results, like hyperparameter tuning and dynamic ensemble selection that did not help. These are rarely published but prevent wasted effort. It makes the trade-off between false alarms and missed attacks clear for security professionals and illustrates the performance that an ML detector can reasonably be anticipated to offer, particularly at the realistic base rate of benign traffic. It demonstrates to students and teachers how statistical testing, ablation, and cautious data processing transform a classification exercise into reliable proof.

1.9 Research Contribution

This research makes the following contributions:

- **A leakage-safe, imbalance-aware two-stage ensemble** for multi-class flow-based intrusion detection. It reaches macro-F1 0.746 [0.727, 0.763] on the curated CIC-UNSW-NB15 file and 0.687 [0.671, 0.704] on the full release at a 97.5% benign base rate.
- **Evidence that moderate rebalancing beats full rebalancing.** Tempered class weighting ($\alpha = 0.25$) was the largest modelling lever (+0.039 macro-F1, $p < 0.001$), while the standard "balanced" scheme and full SMOTE both performed worse than milder alternatives.
- **Evidence that information beats capacity.** Recovering service identity added +0.098 macro-F1, about twice the gain of the entire modelling stack. Hyperparameter tuning, dynamic ensemble selection, threshold tuning and wider ensembles produced no established gain.
- **A reject class and an end-to-end combiner objective** for the realistic base rate. Together they cut benign false alarms fourfold and add +0.027 macro-F1, both statistically established.
- **A robustness analysis under physically consistent perturbations.** It shows that detection and family attribution degrade differently depending on the threat model, and it introduces misattribution as an attacker objective that evasion-only evaluation cannot see.
- **A transparent, reproducible evaluation protocol** with cross-fitting, a single test-set evaluation and paired bootstrap confidence intervals, with every number traceable to a logged run.

1.10 Thesis Organization

The remaining parts of this thesis are organized into five further chapters. Chapter 2 reviews the previous works on machine-learning based intrusion detection and identifies where it falls short. The proposed model is the subject of Chapter 3. This chapter covers its feasibility and requirements, then its design, construction and training. Chapter 4 turns to testing. It sets out the experimental environment and evaluation method before reporting the results in detail. Standards, ethics, security and the project's practical constraints are

the subject of Chapter 5, along with the timeline. Finally, Chapter 6 summarizes the work, its contributions and limitations, and suggests where future research could go.

1.11 Summary

This chapter explained the problem we are trying to solve. Computer networks get attacked, and we want a system that can spot attacks and say what kind they are. This is hard because some attack types are very rare in the data, and because careless testing can make a system look better than it really is. We also listed our goals and the questions we want to answer. We explained why this work matters and what new things it adds. In the next chapter, we look at what other researchers have already done.

Chapter 2
Literature Review

2.1 Introduction

Chapter 1 argued that a network intrusion detection system has two jobs. It must catch malicious traffic, and it must tell attack families apart even when some families are far rarer than others. This chapter reviews the research on that problem. Section 2.2 surveys the published work, Section 2.3 draws out the research needs that the proposed system sets out to meet, and Section 2.4 summarises the chapter.

2.2 Literature Review

The review is organised by theme rather than by paper. It begins with intrusion detection benchmarks and what their features capture (Section 2.2.1), then covers conventional learners and tree ensembles (2.2.2) and deep and hybrid models (2.2.3). Four themes carry most of the weight for this thesis: class imbalance (2.2.4), feature selection and traffic representation (2.2.5), and hierarchical classification and ensemble combination (2.2.6). Explainability (2.2.7) and adversarial robustness (2.2.8) come last, because both decide how useful a detector is in practice. Each subsection compares existing approaches and weighs what they do well against where they fall short.

2.2.1 Network Intrusion Detection and Benchmark Characteristics

At heart, machine learning for intrusion detection is about linking traffic measurements to security labels. How well that works depends on one thing above all. The measurements have to capture the behaviour that separates normal communication from an attack.

Sommer and Paxson (2010) looked at why strong results in experiments so often fail to carry over to real intrusion detection. They pointed to three problems. Network behaviour varies enormously. Classification errors carry real costs. And predictions are hard to interpret in a security setting. Their analysis leads to an important principle. For a detector to be useful, it has to do two things. It must provide the analysts with useful information about every attack. In the environment for which it was intended, it must also limit the quantity of notifications to a reasonable level.

Benchmark datasets give researchers common ground for studying these issues. Moustafa and Slay (2015) introduced UNSW-NB15, which contains normal traffic alongside several categories of attack. Researchers can use it in two ways. It suits studies of plain detection, and it suits studies that classify attack types. A second benchmark came from Sharafaldin et al. (2018). They laid out a method for building an intrusion detection dataset and describing the attack traffic inside it. That work became the basis of CIC-IDS2017. Together, these efforts widened the range of traffic that detection systems could be tested against.

Still, a dataset's name does not tell the whole story. Which records are chosen, how the features are built, which attack categories are used and how the classes are balanced all change how hard an experiment is. Knowing that traffic is malicious is also a different task from knowing which family it belongs to. A high score on binary detection says little about multiclass performance. That's why research that tries to name attacks shouldn't stop at overall figures. Precision, recall and F1 need to be reported for each class too.

Bertoli et al. (2021), on the other hand, stepped back and took a broader view. They built an end-to-end framework that links traffic processing, machine learning and the running system. This matters because feature extraction and prediction have to work together smoothly inside one system. For the present research, it supports looking closely at how the traffic representation and the classifier's design fit together. For that reason, methods here are compared by what they can do, what they assume and how well they suit the task of recognising attack families. A simple ranking of reported accuracies would miss most of that.

2.2.2 Conventional Machine Learning and Tree Ensembles

Tree ensembles are a strong starting point for detection based on tabular flow features. Random Forest combines many randomised decision trees, so no single tree's split has too much say (Breiman, 2001). XGBoost takes a different route. XGBoost grows its trees in sequence, each one correcting the last. Regularisation and a set of computational shortcuts keep the process in check (Chen & Guestrin, 2016). Random Forest and XGBoost can both pick up non-linear patterns in traffic data. They just get there by different routes. Because of that, it pays to study each model class by class, and to see where each one stumbles, before deciding how to combine them.

Thockchom et al. (2023) proposed an ensemble of three simple models, Gaussian naive Bayes, logistic regression and a decision tree. A stochastic gradient descent meta-classifier sits on top. The study also used chi-square feature selection. It evaluated both binary and multiclass classification on KDD Cup 1999, UNSW-NB15 and CIC-IDS2017. The work shows that fairly lightweight classifiers can still make a useful ensemble. That is relevant for detection systems that have to balance accuracy against computing cost.

Using a mix of algorithms sounds like a recipe for diversity, but it isn't always one. Different classifiers can still make the very same mistakes. This happens most often with attack families that look alike in the feature space. Put such models together and the overall score may rise. The hardest minority classes, though, may hardly improve at all. So a good ensemble needs more than strong members. Its members also have to cover for each other's weak spots. tuned single classifiers that use the same features and the same treatment of classes

Injadat et al. (2021) took a wider view with a multi-stage, optimised framework for network intrusion detection. Their study shows what can be gained by handling several pipeline choices together, from how the inputs are prepared to how the learning methods are set up. There is a catch, however. If many parts change at the same time, it is hard to tell which one actually made the difference. Only controlled comparisons can sort that out. For this reason, a hierarchical ensemble should be tested against well-tuned single classifiers. Those classifiers should use the same features and treat the classes in the same way.

2.2.3 Deep Learning and Hybrid Architectures

Rather than depending on manually created features, deep learning attempts to learn meaningful information between representations on its own. Khan (2021) has shown how HCRNNIDS that implements a hybrid network joining convolutional and recurrent layers can be used for intrusion detection. Qazi et al. (2023) offered another hybrid design, called HDLNIDS. Taken together, the two studies show how neural building blocks can be combined to process network data and pick up complex links between inputs.

Whether these designs fit depends on the shape of the data. Convolutional layers are good at local patterns. Recurrent layers are built to follow relationships across inputs that come in order. The order of the feature columns in compiled flow records may not always

correspond to the sequence in which network events occurred. Any neural component should be justified by the true meaning of its inputs instead of the complexity of its architecture.

An explainable ensemble that integrates three lengthy short-term memory classifiers via a meta-learner was proposed by Ben Ncir et al. (2024). Their approach integrates interpretation, ensemble prediction, and representation learning into a single system. That joined-up view is helpful, because a detector needs to classify traffic and also let people examine why it made each call.

For this research, the deep learning studies are an important point of comparison. They show other ways to represent traffic and combine models. What they do not show is that a more complex architecture will recognise rare attacks any better. A fair comparison has to look at performance on each class, at computing cost and at the structure of the traffic data. Those points support testing whether engineered flow features and tree-based classifiers can name attacks effectively, with each decision stage easy to identify.

2.2.4 Class Imbalance and Rare Attack Recognition

Class imbalance causes trouble in two places, during training and when the results are read. If some attack families turn up only rarely, a classifier can score high accuracy and still do a poor job of spotting them. An average weighted by class size has the same problem, since the common classes swamp it. Macro-averaged F1 gets around this by giving every class an equal say in the final figure. That makes it a good fit when classes are so unevenly sized. Even so, it shouldn't be read on its own. Class support, precision and recall belong next to it, because scores built on very small classes can swing quite a lot.

Chawla et al. (2002) has shown a new technique named Synthetic Minority Over-sampling Technique, better known as SMOTE. This method makes new samples instead of copying rare examples over and over, by blending pairs of neighboring samples. A practical idea. How well it works, though, depends on the shape of the feature space. If the neighbours sit in regions where classes overlap, the new points can end up ambiguous. Network traffic adds one more risk. A blend of feature values can be perfectly valid on paper and still describe traffic that no real network would ever produce.

Recent intrusion detection studies tend to pair balancing with other parts of the pipeline. Dasari et al. (2026) brought together K-means SMOTE, Boruta feature selection and stacking in ASENIDSB. Nassreddine et al. (2025) mixed SMOTE-Tomek with feature selection and ensemble learning. Both treat minority representation as tied up with feature quality and classifier design, which makes sense.

Mixing so many parts raises a fair question, though. Where does the gain actually come from? Some of it might be the balancing. Some might be feature selection, or the way the models are combined. Or maybe it only shows up when they all work together. For attack attribution this matters a great deal. A gain that lands mostly in one common attack category says nothing about rare families. So balancing methods should be judged by how each class's performance changes, and by the trade-off between precision and recall that comes with it.

Adjusting the loss is another route, one that doesn't need any new examples at all. Cui et al. (2019) give rare classes more importance during training, but only a moderate amount. Lin et al. (2017) make the model pay less attention to easy examples and more to hard ones. Both ideas came out of computer vision, so there's no guarantee they carry over to network intrusion detection. What makes them relevant is the idea behind them, that the treatment of each class can be tuned gradually rather than switched on or off. Which leads to a pretty obvious question. Could a moderate amount of weighting do better than both extremes? Not treating every class the same, but not rebalancing heavily either. Something in between, that catches more of the rare classes without wrongly labelling too many flows along the way.

2.2.5 Feature Selection and Traffic Representation

Feature selection tries to keep the measurements that matter while cutting redundancy and computing cost. Emirmahmutoğlu and Atay (2025) put heuristic selection methods side by side with several machine learning classifiers, across a number of intrusion detection datasets. What they found is that a good feature subset isn't good everywhere. It all comes down to the classifier, and the dataset too. What helps one method might not do a thing for another.

Nassreddine et al. (2025) went a different way. They paired correlation-based selection with an embedded XGBoost procedure. Correlation looks at statistical association. An

embedded method, on the other hand, reflects how one particular learner actually uses its inputs. Used together, they can help trim the feature set down. The subset that comes out, though, still hangs on the selection criteria and on the model.

Dasari et al. (2026) started with Boruta. After that, they joined Extra Trees and XGBoost through a logistic regression meta-model. So the feature assessment feeds right into how the ensemble gets built. Removing features has a general weakness, though. A variable that looks weak on its own might still matter through the way it interacts with others. The opposite problem exists too. Keep lots of closely related measurements and the model gets more complex, without learning much more about how traffic behaves. In the end, feature selection can only be judged by what it does to the actual classification task.

Building new features is a related problem, but not the same one. No amount of column selection can bring out a relationship the columns don't express well in the first place. In flow-based intrusion detection, things like the ratio of traffic in each direction, bytes per packet, how much the timing varies, and protocol or service information may capture behavior that raw totals miss. Whether they really do is a hypothesis. It has to be tested.

Service information comes with a catch of its own. So much depends on the way a network is set up. Something that works well in one place might not hold up in another. Change the services, the applications, or just the traffic patterns, and it can lose its edge. All of this is a good reason to look at feature construction and feature selection together. It also hints that some of what looks like a classifier's limitation may really come from a representation that doesn't capture the traffic's behavior well enough.

2.2.6 Hierarchical Classification and Ensemble Combination

Hierarchical classification and ensemble combination sound alike, but they tackle different parts of intrusion detection. A hierarchy breaks the decision into linked tasks. First, is this traffic malicious? Then, which attack family is it? An ensemble does something else. It combines the predictions of several models, which may all be working on the same task. Stacking is one kind of ensemble, where an extra learner takes the component predictions and combines them.

Keeping these apart matters when reading about hybrid systems. The multi-stage framework of Injadat et al. (2021), and the stacking approaches of Thockchom et al. (2023)

and Dasari et al. (2026), all make a case for integrated designs. Still, each part has to be understood by the job it actually does in the decision. A system with several algorithms in it isn't automatically a hierarchy that splits benign detection from attack-family classification.

For this problem, a hierarchy gives a clean split between detection and attribution. It also lets errors travel. If the first stage calls an attack benign, the second-stage classifier never sees it, so it can't fix the mistake. The reverse can happen too. Benign traffic that reaches an attack-only classifier may end up with an attack label even though none fits. So a second stage that looks strong when tested on its own says little about how the whole system performs.

Dynamic ensemble selection brings a different kind of specialisation. Ko et al. (2008) traced the move from dynamic classifier selection to dynamic ensemble selection, both built on local competence. The appeal is easy to see when different classifiers do well in different parts of the input space. Whether it works, though, depends on those parts holding enough representative examples to judge competence reliably. Rare attack families can make that very hard.

That's why a dynamic method needs to be compared with simpler options, such as plain averaging and fixed weighting. The extra machinery only earns its place if it gives a steady benefit that the strength of its member classifiers can't explain on its own. This is also where class treatment and ensemble diversity meet. Change what the training focuses on, and both the individual predictions and the room for complementary combination can change with it.

Uddamari and Sammulal (2025) proposed CyberAdaptAI. It joins adaptive ensemble weighting with drift detection and retraining, in a streaming framework. That's a different goal from local selection in a fixed classification setting. Temporal adaptation reacts to change over time. Local selection reacts to differences between individual observations. Each should be judged by its own assumptions.

Overall, the literature supports looking for complementary classifiers and sensible ways to combine them for attack attribution. It also shows that adding models, or bringing in dynamic selection, doesn't promise an improvement. The real question is which way of

combining them helps the target attack families, given the representation and the class distribution in use.

2.2.7 Explainability and Operational Interpretation

A detector's prediction is far more useful when an analyst can look at the evidence behind it. Lundberg and Lee (2017) introduced SHAP, a framework for additive feature attribution. It gives each feature a share of the credit for an individual prediction. That makes it a good basis for studying which traffic measurements push a model's output one way or the other. These attributions describe how the fitted model behaves under a particular explanation procedure, though. On their own, they prove neither cause nor that the prediction was right.

Ahmed et al. (2024) went further with an explainable hybrid ensemble. It combines stacking, Bayesian model averaging and a conditional ensemble mechanism, and uses SHAP and LIME to interpret the results. Ben Ncir et al. (2024) also built explainability into an ensemble deep learning system. Clearly, explanation methods are already part of intrusion detection research.

So what explainability adds depends on how it helps the investigation. Feature attributions can be used to compare predictions for common and rare attacks. They can help examine flows that were misclassified. They can also show whether the model relies on traffic features that make sense for the environment it's meant for. They're worth most when they answer a specific question, and least when they're just shown as general feature-importance plots.

How useful a detector is in practice also comes down to how many alerts it raises and how reliable its predictions are. Sommer and Paxson (2010) stressed that the cost of errors, and how varied network behaviour can be, make real-world intrusion detection hard. When benign traffic dominates, even a small false-positive rate on benign flows can mean a lot of alerts. Detailed attack labels only help if they're dependable enough to investigate. That ties explainability to threshold choice, to precision, and to looking at errors class by class.

2.2.8 Robustness and Synthesis of Existing Research

Robustness research asks a simple question. Will a model still behave usefully if its inputs change, or the conditions it runs in? That's the question. Vitorino et al. (2023) looked at it through a review of realistic adversarial attacks and defences for intelligent network intrusion detection. One thing they kept coming back to was the limits that network communication and protocols put in place. That matters a lot for classifiers built on features. Nudging numbers independently may produce a combination that no real traffic could ever create.

Random noise, carefully optimised evasion and attacks borrowed from another model each test something different. So a result under one of these conditions should be read only within that condition. With a hierarchical detector, robustness also has more than one way to go wrong. An altered attack might slip past completely. Or it might be caught but placed in the wrong family. For security analysis, those two outcomes mean very different things.

Put side by side, the reviewed studies point in several directions that fit together. Tree ensembles and neural ensembles offer different ways to combine predictions. Balancing methods shape which examples the model pays attention to while learning. What information the classifier actually gets to work with? That comes down to feature selection and feature construction. Hierarchical decisions, meanwhile, can help keep detection and attribution organised. And explanation methods can help look into how the result behaves.

What remains open for this thesis is how all these choices play out together. The goal is to recognise attack families reliably while still handling benign traffic acceptably. A component that has worked well elsewhere is a good reason to try it. It doesn't decide, though, which combination works best for this particular traffic representation and class distribution.

2.3 Problem Analysis

The core problem is a hard one. How do you name attack families reliably without flooding analysts with false alerts? Several connected causes make this difficult.

The first is that training evidence is spread unevenly across the classes. A model can learn the dominant benign and attack patterns well and still not see enough examples to tell the

rare families apart. Giving minority classes more influence can make the model more sensitive to them. Push too far, though, and their decision regions start spilling into areas that belong to other classes. That is why the work of Chawla et al. (2002), Cui et al. (2019) and Dasari et al. (2026) points toward a different question. How much class correction is the right amount?

The second cause is overlap in the traffic representation itself. Different attack mechanisms can produce similar packet counts, durations or rates. Meanwhile, one attack family can show up in several different forms. If the measurements don't separate the categories well enough, no classifier can tell them apart reliably. The feature selection studies of Emirmahmutoğlu and Atay (2025) and Nassreddine et al. (2025) show how much informative inputs matter. Removing variables, though, is not the same as building meaningful new relationships. So the quality of the representation has to be checked before blaming every leftover error on a model that isn't complex enough.

A third difficulty sits between detection and attribution. Knowing that traffic is malicious is one thing. Knowing which attack family it belongs to is another. A flat classifier learns both inside one shared decision process. A hierarchy hands them to separate stages instead. That allows each stage to specialise, but any mistake in the first stage limits what the second can do. To understand the full system, one has to look at how missed attacks, benign false positives and wrong family labels each feed into the final output.

The fourth difficulty is knowing when a more complex ensemble is actually worth it. Stacking and dynamic selection both offer ways to combine classifiers. Combining only pays off, however, when the members make different mistakes and their competence can be estimated reliably (Ko et al., 2008; Thockchom et al., 2023). If the errors are strongly correlated, adding more models doesn't help much. Sparse neighborhoods around minority classes can also make local competence estimates shaky. All this is a good reason to study class treatment and model combination together. Changing what the training emphasizes can shift both how each model performs and how diverse the ensemble is.

What this adds up to is a clear research need. It calls for a systematic look at how traffic representation, minority-class treatment and hierarchical ensemble decisions together affect performance on attack families. Any contribution should rest on evidence. That evidence should show when the combination improves the target task, which classes gain, and which

limits remain. Explainability and well-defined robustness experiments can add supporting evidence about how the model behaves. With the reasoning for the proposed system now in place, its detailed architecture and implementation are left to the next chapter.

2.4 Summary

This section clarifies Network intrusion detection gains from three things, namely informative traffic representations, the right treatment of classes, and well-chosen combinations of classifiers. It also shows why overall accuracy isn't enough, either for judging how well rare attacks are recognized or for judging whether the final alerts are any use. The problem analysis then pinned down four challenges that feed into each other. These are class imbalance, overlapping features, errors carried through a hierarchy, and uncertainty over whether ensemble members really complement one another. Taken together, these findings make a case for a hierarchical ensemble in which representation and class treatment are studied alongside the way models are combined. The next chapter sets out the proposed model, and the methodology used to test how these pieces relate.

Chapter 3

Proposed Model

3.1 Introduction

This chapter introduces the proposed intrusion detection model and shows how it tackles the problems raised so far. Before any design work, though, there is a basic question: can this actually be built? Section 3.2 looks at this question in three different ways. First, it checks whether the technology is up to the task. It then considers whether the cost is reasonable. Finally, it asks if the system would actually work in practice. Once it is clear that the idea is workable, Section 3.3 turns to what the system must actually do. It also explains how well the system needs to perform for it to be useful. The research approach comes next. Section 3.4 explains the methodology and why machine learning, and tree-based models in particular, suit this problem. Section 3.5 then walks through the system's architecture piece by piece. How it was all built, trained and tested is the focus of Section 3.6, while Section 3.7 covers the tools behind the work. Section 3.8 wraps up.

3.2 Feasibility Analysis

3.2.1 Technical Feasibility

None of these methods had to be built from scratch. Python already offers mature, open-source versions of all of them (Chen & Guestrin, 2016; Lemaître et al., 2017; Pedregosa et al., 2011).

Hardware was the next worry, and it turned out to be a small one. Tree ensembles run comfortably on an ordinary multi-core processor. They don't need a graphics card. They don't even need the features to be rescaled first. That makes them a natural fit for tabular flow data, even at the scale of this project, which reaches 3.5 million records.

Then there was the data itself. Collecting network traffic usually means setting up packet capture tools or generating traffic in a lab. Here, none of that was necessary. The dataset is public, and it comes already converted into flow.

3.2.2 Economic Feasibility

Money was never a real obstacle. Every piece of software used here is free and open source.

The datasets cost nothing either, since their creators released them publicly for research. And instead of renting cloud servers, all the experiments ran on an ordinary personal computer. In the end, the project needed nothing beyond what was already at hand. The main cost was researcher time. For an organization adopting the approach, the cost of deployment would be low compared with commercial intrusion detection products. Flow exporters are built into most network equipment, and the trained model is a set of tree ensembles that classify flows on ordinary server hardware. The solution is therefore economically viable both as a research project and as a potential deployment.

3.2.3 Operational Feasibility

Operationally, the system fits naturally into existing security monitoring practice. Network flow records are already collected by many organizations for monitoring and capacity planning, and the system's output is the familiar output of an IDS: an alert carrying an attack label. The two-stage design has an operational advantage: the gate alone provides a binary "attack or not" signal, and the family label is added only for flows that raise an alert. The analysis in Chapter 4, however, shows realistic limits. In real traffic, where benign flows vastly outnumber attacks, some alerts will still be false alarms. The attack-type labels are also less dependable than the basic decision of whether an attack is happening at all. So the system works best as a helper for human analysts. It can point them toward suspicious traffic, but it shouldn’t be trusted to block traffic on its own.

3.3 Requirement Analysis

The requirements below were derived from the problem statement and research objectives (Chapter 1). Functional requirements describe what the system must do (Table 3.1). Non-functional requirements describe the qualities it must have (Table 3.2). Each requirement is stated so that it can be checked, and the last column records how it was verified in this work.

Table 3.1 Functional requirements

ID	Requirement	Verification
FR1	The system shall accept a network flow record described by CICFlowMeter statistics, destination port and protocol.	Input schema of the implemented pipeline (Section 4.3)
FR2	The system shall classify each flow as benign or malicious (detection).	Stage-A gate; detection recall and precision reported in Chapter 4

ID	Requirement	Verification
FR3	For each flow classified as malicious, the system shall assign one of nine attack families (attribution).	Stage-B ensemble; per-class results in Chapter 4
FR4	The system shall be able to withdraw an alert when the second stage judges a flow to be benign traffic that passed the gate.	Reject output of Stage B (Section 3.6.4)
FR5	The system shall derive the engineered behavioural and service-identity features automatically from the input record.	Feature construction step (Section 3.6.3)
FR6	The system shall support retraining on new labelled data using the same pipeline.	Scripted end-to-end training procedure (Section 3.6.5)
FR7	The system shall report evaluation results per class, not only overall.	Classification reports and confusion analysis (Chapter 4)

Table 3.2 Non-functional requirements

ID	Category	Requirement	Verification
NFR1	Detection quality	Rare attack families must be detected; macro-F1 is the primary metric and must improve significantly on single-model baselines.	Paired bootstrap tests (Section 4.5)
NFR2	Evaluation validity	No test-set information may influence training or model selection; duplicates must not cross the train–test split.	Pipeline ordering and single test-set evaluation (Section 3.6.6)
NFR3	Reliability of claims	Every reported score must carry a 95% confidence interval, and every claimed improvement a paired significance test.	Bootstrap procedure (Section 4.4)
NFR4	Reproducibility	Experiments must be repeatable from the code with fixed random seeds and logged outputs.	Seeds, logs and cached predictions (Section 4.2)
NFR5	Efficiency	Training must complete on commodity hardware without a GPU; classifying a flow must require only a fixed, small number of model evaluations.	Run times in Section 4.2; inference path in Section 4.3
NFR6	Scalability	The pipeline must handle the full 3.5-million-flow release.	Full-release experiments (Section 4.5)
NFR7	Robustness	Performance under small input perturbations must be measured and reported.	Robustness simulation (Section 4.5)
NFR8	Maintainability	Components (features, gate, ensemble, combiner) must be separable so that each can be replaced and tested independently.	Modular design (Section 3.5) and component ablations

3.4 Research Methodology

This research follows an experimental, quantitative machine-learning methodology. The problem was framed as supervised multi-class classification of network flows into benign traffic and nine attack families. Classical supervised learning with tree-based ensembles was chosen over deep learning for three reasons. First, flow records are tabular data with heterogeneous, heavily skewed features, a setting where gradient-boosted and bagged tree ensembles are consistently strong and need no feature scaling. Second, tree ensembles train quickly on a CPU, which made it possible to run the many controlled experiments this methodology requires. Third, their behaviour under class weighting and resampling is well understood, which was essential because class imbalance is the central difficulty of the problem. The methodology consists of six steps, shown in Figure 1.1 (Chapter 1).

1. Data collection. The primary data is the CIC-UNSW-NB15 dataset in its curated form (447,915 flows) and its full release (3,540,241 flows) (Mohammadian et al., 2024). Three further public datasets were used only as external reference points. No new traffic was generated and no human subjects were involved.

2. Data preprocessing. Data quality was checked, exact duplicates were removed, uninformative and redundant features were pruned without using labels, and the data was split into stratified training and test sets. Two groups of features were then added: scale-free behavioural ratios and service-identity features. Every step that depends on labels runs after the split, on training data only.

3. Model training. Baseline classifiers were trained first to establish reference performance. The proposed two-stage system was then developed iteratively: each design change (class weighting, the hierarchy, the ensemble, new features, the reject class, the combiner objective) was introduced as a controlled experiment and kept only if it produced an established improvement on held-out data. Ensemble combination weights were fitted on cross-fitted out-of-fold predictions so that no model was evaluated on data it had trained on.

4. Performance evaluation. Macro-F1 was the main yardstick for comparing models. It gives every attack type equal weight, so rare attacks count as much as common ones. Other measures filled in the rest of the picture. Accuracy, precision, recall and ROC-AUC were all reported, along with results for each class. Each score carries a bootstrap confidence interval, and each comparison was tested with a paired bootstrap, so that improvements

could be separated from random variation of the test sample.

5. Robustness analysis. Because the template's explainability step was out of scope for this thesis (Chapter 6), the final model was instead analyzed for reliability. Test flows were perturbed in physically consistent ways, by random noise and by targeted attacks, to measure how detection and family attribution degrade.

6. Result analysis. Results, including negative ones, were interpreted against the research questions. The causes of remaining errors were examined through label-ambiguity and nearest-neighbour class-overlap analyses, and the strengths and limitations of the approach were assessed.

This methodology is appropriate because the research questions are comparative ("does component X improve detection, and under what conditions?"). Comparative questions are answered most convincingly by controlled experiments in which a single factor changes and the difference is tested statistically.

3.5 System Design and Architecture

The system is built in layers, one feeding into the next, as Figure 3.1 shows. A raw flow record goes in at the top. What comes out is one of two things. Either the flow is judged benign and no alert is raised, or an alert goes up naming one of nine attack families.

The main idea is to split the work in two stages. A flat classifier tries to answer everything in one go. This design asks two separate questions instead. First, is this flow malicious? And if it is, which kind of attack is it?

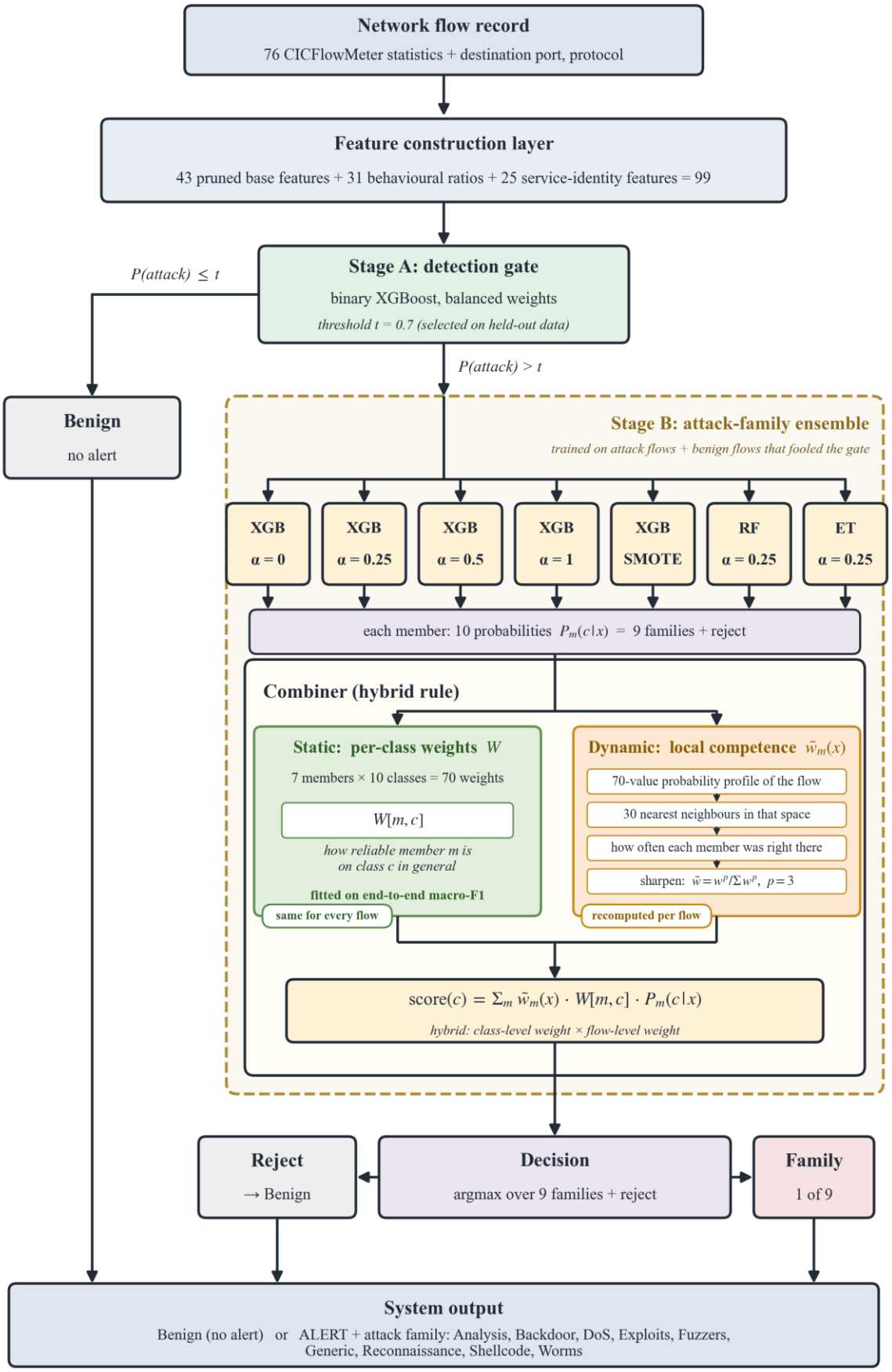


Figure 3.1 Architecture of the proposed two-stage intrusion detection system

3.5.1 Components

Input layer. Each input is a bidirectional network flow record as produced by CICFlowMeter: 76 statistics describing packet counts, sizes, timing, flags and activity, plus the flow's destination port and transport protocol.

Feature construction layer. This layer applies the label-free transformations learned from the training data. It keeps the informative base features selected by the variance and correlation pruning (43 on the full release, 44 on the curated file), computes 31 scale-free behavioural ratios from the raw statistics and encodes the destination port and protocol as service-identity features (25 on the full release, 26 on the curated file). The final system therefore uses 99 features.

Stage A, the detection gate. Every flow first meets the gate, a binary XGBoost classifier. Its job is simple. It looks at the flow and estimates how likely it is to be malicious. If that chance is at or below a threshold t, the gate waves the flow through as benign. Its journey ends there. Anything above t is a different story. The threshold was chosen on held-out data; the final system uses $t = 0.7$, although every value from 0.3 to 0.7 scored within 0.0034 macro-F1. That flow gets handed on to Stage B for a closer look.

Why keep the gate so simple? Because it only has one question to answer. Benign or attack? It never has to tell one attack family from another. That frees it to learn from the entire training set without getting tangled up in the finer differences between, say, DoS and Exploits.

Stage B, the attack-family ensemble. Seven tree-based classifiers, trained on attack flows (and, in the final version, on benign flows that previously fooled the gate), each output a probability for every class. Most of the team, five members in all, are XGBoost models. What sets them apart is how each one deals with the imbalance between common and rare attacks. Four use tempered class weights, with α set to 0, 0.25, 0.5 and 1. The fifth takes a different route and learns from extra synthetic examples created by capped SMOTE oversampling. The last two members break from the XGBoost pattern entirely. One is a random forest, and the other is an extra-trees model. The members are deliberately diverse in how they trade precision against recall on rare families.

Combiner. The members' probabilities are fused by a hybrid rule that multiplies two kinds of weight (Figure 3.1). The first is a learned per-class weight matrix W, with one weight per member and class, so that each member counts most for the families where it is reliable

in general; W is fitted to maximise the macro-F1 of the whole pipeline. The second is a per-flow competence weight in the style of dynamic ensemble selection (Cruz et al., 2018): the flow's 70 member probabilities are used to find its 30 nearest neighbours among held-out training flows, each member is scored by how often it was right on those neighbours, and the scores are sharpened with an exponent p = 3. The fused score of class c is the sum over members of competence × W[m, c] × the member's probability for c. Using W alone gives a static combiner and using the competence weight alone gives a dynamic one; the hybrid was chosen on held-out data from 21 candidate rules that included both.

Decision and output layer. The class with the highest fused score is selected. If it is one of the nine families, an alert carrying that family label is raised. If it is the reject class, the alert is withdrawn and the flow is reported as benign.

3.5.2 Data Flow and Design Rationale

Data moves through the system in one direction only, from each layer to the next. No layer ever reaches back. Each one simply takes what the previous layer hands it and does its own job. This keeps the parts independent of each other. Any single component can be retrained, swapped out or tested by itself without disturbing the rest (NFR8). The design responds directly to the problem analysis:

- **Hierarchy.** Separating detection from attribution lets the gate be optimized for catching attacks, while the family classifier focuses entirely on the harder problem of telling attacks apart, without the 80–97% benign majority dominating its training.
- **Imbalance-diverse ensemble.** No single weighting scheme suits all families: heavier weighting improves recall on rare families but costs precision on common ones. Combining members across the weighting spectrum lets the combiner choose the right trade-off per family.
- **Reject class.** At the realistic base rate, even a highly accurate gate passes thousands of benign flows to Stage B. Letting Stage B recognize and return them reduces false alarms without retraining the gate.
- **End-to-end objective.** Fitting the combiner on the deployment objective (macro-F1 of the whole system) rather than on Stage B's own objective aligns every component with what the user actually experiences.

3.6 Implementation and Simulation

This section explains how the proposed intrusion detection system was built, trained and tested, in enough detail for the work to be replicated. Development took place in two settings. Most design decisions were made on the curated CIC-UNSW-NB15 file of 447,915 network flows (Mohammadian et al., 2024). The final model was then retrained and evaluated on the full release of the same dataset, which contains 3,540,241 flows, 97.5% of them benign. This proportion is much closer to the conditions a real network monitor faces, and it is the setting in which the final results are reported.

3.6.1 Datasets

The primary dataset, CIC-UNSW-NB15, was published by the Canadian Institute for Cybersecurity in 2024. It re-processes the traffic of the UNSW-NB15 benchmark (Moustafa & Slay, 2015) with the CICFlowMeter flow extractor (Lashkari et al., 2017). Each record is a single bidirectional flow described by 76 numeric statistics: packet and byte counts in each direction, packet-length statistics, inter-arrival times, TCP flag counts, window sizes, and active and idle periods. Each flow is labelled either Benign or as one of nine attack families. The curated version is distributed as a feature file and a label file aligned by row position. Before any processing, the label encoding was checked against the text labels in the full release, and all 447,915 rows matched the mapping in Table 3.3.

Table 3.3 Class distribution of the curated CIC-UNSW-NB15 file (before deduplication)

Label	Class	Flows	Share (%)
0	Benign	358,332	80.00
1	Analysis	385	0.09
2	Backdoor	452	0.10
3	DoS	4,467	1.00
4	Exploits	30,951	6.91
5	Fuzzers	29,613	6.61
6	Generic	4,632	1.03
7	Reconnaissance	16,735	3.74
8	Shellcode	2,102	0.47
9	Worms	246	0.05
	Total	447,915	100.00

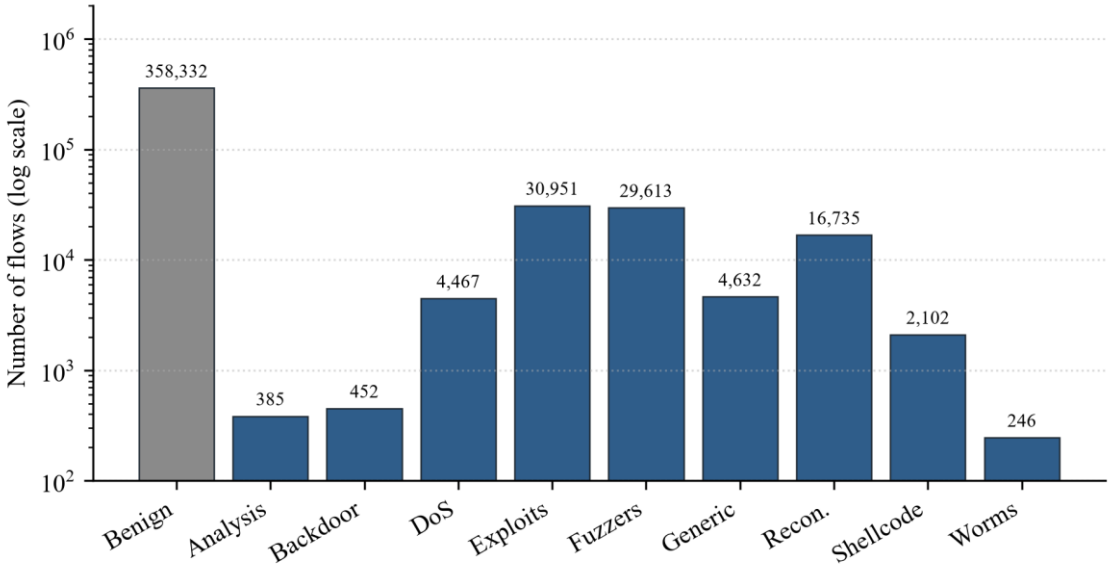


Figure 3.2 Class distribution of the curated file on a logarithmic scale

The data is severely imbalanced (Table 3.3, Figure 3.2). Worms is the rarest family of all, with just 246 flows, and benign traffic outnumbers it roughly 1,457 to one. That gap creates a trap. Imagine a model that simply calls every flow benign. It would catch no attacks at all, yet it would still score 80% accuracy, so accuracy on its own says very little about how good the system really is. The full release makes the picture even bigger. Beyond the curated file, it ships with the complete CICFlowMeter output, 3,540,241 flows in total. Each one carries the same 76 statistics, along with identity fields such as the flow identifier, addresses, ports, protocol and timestamp, plus a text label. Matching showed that the curated file is an exact subset of it: every attack flow is retained, and benign flows are downsampled about tenfold. The full release was used for two purposes: to supply each flow's destination port and protocol, which the curated file omits, and to train and test the final system at the realistic benign share of 97.5%.

Three further datasets served only as external reference points, never for model development or selection: the original UNSW-NB15 files (Moustafa & Slay, 2015), which describe the same traffic with a different extractor; CIC-IDS2017 (Sharafaldin et al., 2018), with 2,830,743 flows of different traffic; and the network subset of TON_IoT (Moustafa, 2021), with 211,043 flows.

3.6.2 Data Preprocessing

The pipeline was designed so that no information from the test data could reach the model during training. Such leakage is a well-documented source of over-optimistic results in security ML (Arp et al., 2022; Kaufman et al., 2012). One rule governed the ordering: any

step that uses class labels, or that learns statistics from the data, runs only after the train–test split and only on the training portion. Table 3.4 lists the steps in the order they were applied.

Table 3.4 Preprocessing steps in execution order (curated file)

Step	Operation	Uses labels?	Result
1	Data quality check (missing values, infinities, data types)	No	No missing or infinite values; all 76 features numeric
2	Removal of exact duplicate records	Only as part of the duplicate key	447,915 → 306,173 flows (141,742 removed, 31.6%)
3	Removal of constant (zero-variance) features	No	76 → 67 features
4	Removal of redundant features (absolute Pearson $r \geq 0.95$)	No	67 → 44 features
5	Stratified train–test split, 80% / 20%	For stratification only	244,938 training / 61,235 test flows
6	Feature engineering (Section 3.6.3)	No	44 → 75 → 101 features
7	Class weighting and oversampling (Section 3.6.5)	Yes	Training data only

Duplicate removal. Nearly a third of the curated records (141,742) were exact copies of another record. If duplicates are kept, copies of the same flow can land on both sides of the split, and the test score partly rewards memorisation. The problem has been recognised since the KDD Cup 99 benchmark (Tavallaee et al., 2009). Duplicates were therefore removed before the split. A dedicated experiment (Chapter 4) found that keeping duplicates barely changed this pipeline's macro-F1 but severely distorted the nearest-neighbour analyses, so deduplication was kept as a correctness requirement.

Removing uninformative and redundant features. Nine features had the same value in every record, for example several URG, CWR and ECE flag counters and the forward bulk-transfer averages, and were removed. For each remaining pair of features with an absolute Pearson correlation of at least 0.95, one was removed, leaving 44 base features. Neither step uses labels, so both can safely run before the split. A follow-up check confirmed that pruning merges no previously distinct flows.

Supervised feature selection. Mutual-information ranking followed by recursive feature elimination with cross-validation was also tested, fitted on training data only (44 → 36 features). It gave no measurable improvement (for example, XGBoost macro-F1 0.5807 → 0.5800), so it is not used in the final system.

Scaling. Tree models split on thresholds and are unaffected by monotonic rescaling, so the classifiers use unscaled features. A median/IQR-based robust scaler, fitted on training data only, was used solely for the distance-based components (the neighbourhood combiner and the diagnostic analyses), because the flow statistics are heavily skewed.

3.6.3 Feature Engineering

Behavioural ratio features. Most CICFlowMeter statistics are absolute quantities that grow with the size of a flow. The engineered features re-express them as size-independent ratios that describe how a flow behaves rather than how large it is. Each is computed from a single flow's own values. None uses the label or other records, so they cannot leak information. Denominators are offset by 10^{-6}, and undefined results are set to zero. Table 3.5 summarises the 31 features. Together with the base features, they form the 75-feature DERIVED representation. Appendix A lists every formula.

Table 3.5 Engineered behavioural features

Group	No.	What it captures
Direction asymmetry	4	Forward-to-backward ratios and normalised differences of packets and bytes (one-sided scans and floods versus two-way conversations)
Bytes per packet	4	Average packet size in each direction and their ratio
Timing regularity	4	Coefficient of variation and relative range of inter-arrival times (automated traffic is unusually regular)
Length regularity	5	Coefficient of variation and relative range of packet lengths
TCP flag fractions	6	Share of packets carrying FIN, SYN, RST, PSH and ACK; SYN-to-FIN ratio
Header overhead	3	Header bytes per packet in each direction; payload share of forward bytes
Activity profile	4	Active and idle time relative to duration, their ratio, time per packet
Window ratio	1	Ratio of initial TCP window sizes in the two directions
Total	31	

Service-identity features. The curated file does not record which port or protocol a flow used, although the targeted service is strongly linked to the attack family. This information was recovered from the full release by locating each curated flow through an exact match of all 76 feature values. A port was used only when all matching records agreed, which held for 94.1% of deduplicated flows. The rest were marked "ambiguous". The protocol was recovered for every flow. The port was then encoded as 26 features (Table 3.6), giving 101 features on the curated file. In the full release, port and protocol are recorded directly. Pruning that larger file left 43 base features, for 99 in total.

Table 3.6 Service-identity features

Type	No.	Content
Well-known services	19	FTP, SSH, Telnet, SMTP, DNS, HTTP, POP3, RPC (111), NetBIOS, IMAP, BGP, HTTPS, SMB, Syslog, RIP, PPTP, AIM, BitTorrent, HTTP-alternate (8080)
Port ranges	3	Other well-known (< 1024), registered (1024–49151), ephemeral (≥ 49152)
Other	4	Ambiguous-port indicator, TCP, UDP, raw port number

Two precautions apply. First, the port agreement check was computed over the whole full-release file, including flows that later fell into the test set. It uses no labels and is deterministic, like the label-free pruning above, but it is stated here for transparency. Second, the testbed's traffic generator directs each attack type at particular services, so part of the value of service identity may be specific to this testbed. For the same reason, source IP addresses and packet time-to-live (TTL) values were deliberately excluded. An audit showed they identify which machine generated a flow, which in this testbed is almost equivalent to the label.

3.6.4 Machine Learning Algorithms

Baselines. Three single classifiers were trained as flat 10-class baselines on the 44 base features, all with balanced class weights: a decision tree (Breiman et al., 1984), a random forest of 200 trees (Breiman, 2001) and XGBoost with 400 boosting rounds (Chen & Guestrin, 2016).

Two-stage ensemble. The proposed system (Section 3.5) uses a binary XGBoost gate trained with balanced weights, followed by the seven-member Stage-B ensemble in Table 3.7.

Table 3.7 Stage-B ensemble members

#	Algorithm	Imbalance treatment
1	XGBoost	None ($\alpha = 0$)
2	XGBoost	Tempered class weights, $\alpha = 0.25$
3	XGBoost	Tempered class weights, $\alpha = 0.5$
4	XGBoost	Full inverse-frequency weights ($\alpha = 1$)
5	XGBoost	SMOTE oversampling capped at 2,000 flows per family (Chawla et al., 2002)
6	Random forest, 300 trees	Tempered class weights, $\alpha = 0.25$
7	Extra-trees, 300 trees (Geurts et al., 2006)	Tempered class weights, $\alpha = 0.25$

The tempered weight of family c, with n_c training flows out of N, is

$$w_c \propto (N/n_c)^{\alpha},$$

rescaled so that the weights average 1

With α = 0 all families are weighted equally. With α = 1 the scheme equals the standard "balanced" weighting, which here gives the rarest family more than a hundred times the weight of the largest. Intermediate values give moderate rebalancing.

Reject class. At the realistic base rate the gate's small false-alarm rate still sends thousands of benign flows to Stage B. In the final system Stage B therefore has ten outputs: the nine families plus a reject output meaning "benign flow that passed the gate". It is trained on the 34,101 benign training flows (1.69% of benign training data) that the gate misclassified during cross-validation. A flow assigned to reject is reported as benign.

Combiner. The fused score for class *c* is

$$S(c|x) = \sum mW[m,c] \cdot P_m(c|x)$$

predicted class = argmax_c S(c | x)

where P_m is member *m*'s probability output. *W* was found by coordinate ascent: each weight in turn is set to the value from the grid {0, 0.25, …, 2} that maximizes macro-F1, over three full sweeps. In the final system the objective is the macro-F1 of the whole pipeline, with reject counted as benign. Optionally, member weights are also scaled per flow by their accuracy on the flow's k nearest neighbors in the space of member outputs. This is the overall local accuracy approach to dynamic classifier selection (Woods et al., 1997). Whether to use it was decided on held-out data.

3.6.5 Training Procedure

Table 3.8 Model settings

Model	Settings
XGBoost	400 trees, maximum depth 8, learning rate 0.1, row subsample 0.9, feature subsample 0.9, histogram tree method
Random forest	300 trees in Stage B (200 in the baseline); library defaults otherwise
Extra-trees	300 trees; library defaults otherwise
SMOTE	5 nearest neighbours; families below 2,000 training flows raised to 2,000
Random seed	42 for every split, model and resampling step

Hyperparameters. All XGBoost models share one configuration (Table 3.8). A random search over 40 alternatives gave no statistically established improvement (Chapter 4), so the configuration was kept.

Cross-fitting. If the combiner were fitted on predictions the members made for their own training data, it would learn from over-confident outputs, a problem familiar from stacked generalization (Wolpert, 1992). The training data was therefore divided into five stratified folds. For each fold, the gate and all members were trained on the other four folds and predicted the held-out fold. This produces out-of-fold predictions for every training flow, which were used to fit W, to build the reject set and to serve as the reference set for the neighborhood refinement.

Final refit. Once the combiner was fixed, the gate and all members were retrained once on the complete training portion, and only then applied to the test set.

Imbalance handling. Class weights are applied as per-sample weights. SMOTE runs only inside the training data of the one member that uses it. Neither is ever applied to validation or test data.

3.6.6 Experimental Setup

Table 3.9 Data partitions

Partition	Purpose	Curated file	Full release
Training (80%)	Cross-fitting, combiner fitting, model selection	244,938	2,085,442
— Combiner half	Fitting W and the neighborhood reference set	122,469	1,042,721
— Selection half	Choosing between combiner variants	122,469	1,042,721
Test (20%)	Final evaluation, used once	61,235	521,361

After deduplication and pruning, each dataset was divided as shown in Table 3.9. Every split is stratified by class with a fixed seed, and Figure 3.3 summarizes the complete protocol. The test set played no part in any design decision. Hyperparameters, combiner variant, gate threshold and feature set were all chosen on training-side data, and each experiment scored the test set exactly once. Every component (class weighting, hierarchy, ensemble, engineered features, service features, reject class, end-to-end objective) was evaluated by comparing two pipelines that differ only in that component, using the same splits and seeds and re-running the comparison pipeline in the same experiment. Section 4.4 defines the evaluation metrics and the statistical tests.

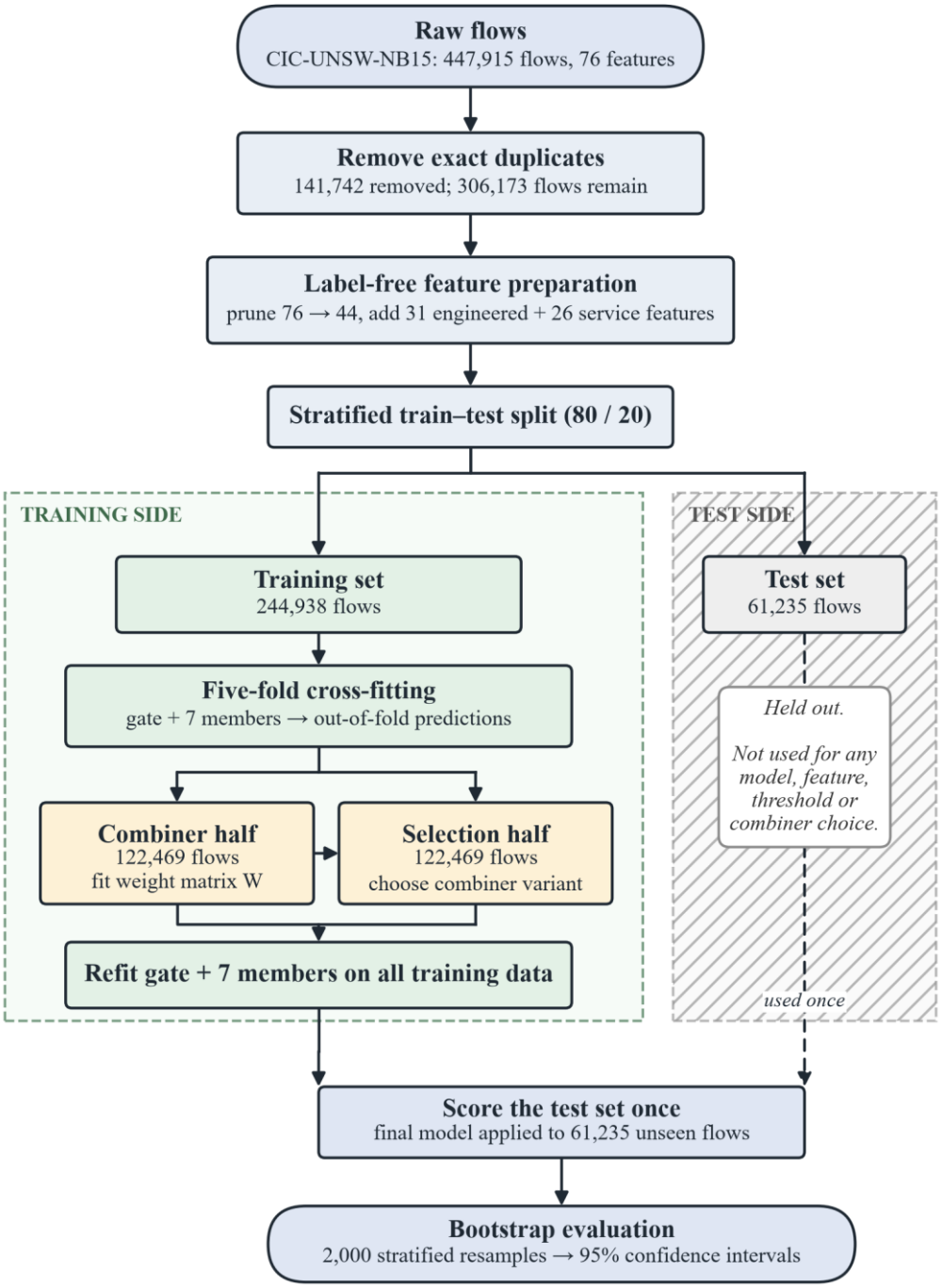


Figure 3.3 Experimental protocol used for every experiment (flow counts for the curated file)

Robustness simulation. The trained system was also tested on perturbed test traffic, following the principle that adversarial changes to network flows must remain physically realizable (Apruzzese et al., 2022). Features were divided into three groups. The first can be changed freely by an attacker (timing, padding, sizes). The second is costly to change (packet, byte and flag counts, which were given half the budget). The third cannot change without abandoning the attack (service, protocol, destination port). The budget was

measured in units of each feature's training interquartile range. Changing one feature of a flow can quietly break others that depend on it. So after every perturbation, those dependent features were recomputed, from the CICFlowMeter rates to the engineered ratios. That way, each modified flow still made sense as a real piece of traffic. Three kinds of threat were then put to the test. The first was plain random noise with no attacker behind it. The second was a targeted white-box attack, aimed at sneaking malicious flows past the gate (evasion). The third was subtler. Here the attacker lets the alert fire but tries to change which attack family the system reports (misattribution).

3.7 Tools and Technologies Used

Python was the natural choice for building the system. It is the leading language for applied machine learning, and for good reasons. Its scientific ecosystem is mature, and every algorithm this project needed already existed as a well-tested, open-source library. Table 3.10 walks through each tool and why it earned a place. For the exact versions, and the hardware everything ran on, see Section 4.2.

Table 3.10 Tools and technologies

Tool	Version	Role in this work	Reason for selection
Python	3.12.8	Implementation language	Standard language for ML research; all required libraries available
pandas	2.2.2	Loading CSV files, deduplication, tabular processing	Efficient columnar operations on millions of rows
NumPy	2.2.6	Numerical arrays, engineered features, caching predictions	Fast vectorised arithmetic; compressed array storage
scikit-learn (Pedregosa et al., 2011)	1.6.1	Decision tree, random forest, extra-trees, splitting, cross-validation, scaling, nearest neighbours, metrics	Reference implementations with a consistent interface and well-tested metrics
XGBoost (Chen & Guestrin, 2016)	3.3.0	Gate and five Stage-B members	State-of-the-art gradient boosting for tabular data; native per-sample weights; fast histogram training on CPU
imbalanced-learn (Lemaître et al., 2017)	0.14.2	SMOTE oversampling	Standard, scikit-learn-compatible resampling library
SciPy	1.13.1	Rank correlations and statistical tests in the analysis	Standard statistical routines
Matplotlib	3.9.2	Figures and charts	Publication-quality plotting with full control over layout

Tool	Version	Role in this work	Reason for selection
Jupyter notebooks	JupyterLab 4.3.4	Exploration and early-stage models	Interleaves code, evidence and narrative, supporting the evidence-first workflow
Conda (Miniconda)	24.11.3	Isolated environment with fixed package versions	Reproducibility across runs

All tools are free and open source, which supports the economic feasibility of the project (Section 3.2) and allows others to reproduce the work without license costs.

3.8 Summary

This chapter presented the proposed model. First came the question of feasibility, and the answer was encouraging. Everything the solution needed was within easy reach. The data was free, the tools were open source, and an ordinary computer was enough to run it all. Next, the requirements were pinned down, seven functional and eight non-functional. Most of them come back to a few core goals. The system should catch attacks and name them correctly, and it should be evaluated without leakage in a way others can repeat. The methodology then set the working rhythm. Tree ensembles were developed through a cycle of controlled experiments, with each round building on what the last one revealed. The architecture combines a detection gate, an imbalance-diverse family ensemble, a learned combiner and a reject class. The implementation section then filled in the practical details. It covered the datasets and the leak-free way they were prepared. It described the engineered and service-identity features, and it explained how the models were trained and tested. After that, the tools section showed why each piece of software was chosen. With the design now complete, the next chapter moves to setup and testing, and it reports what the experiments revealed.

Chapter 4

Implementation and Testing

4.1 Introduction

With the design in place, this chapter puts the system to work and looks closely at how it performs. It starts with the setting. Section 4.2 describes the hardware and software behind the experiments, along with the steps taken so that anyone could repeat them. Section 4.3 then follows a flow's journey through the system, from raw data all the way to a prediction. Before any results can be trusted, though, the measuring stick has to be clear. That is the job of Section 4.4. It defines the evaluation metrics and the statistical tests used to tell a real improvement from a lucky one. The results themselves arrive in Section 4.5, which covers a lot of ground. It begins with how the baseline models performed and what each component added. It then shows how the final system copes with realistic traffic. From there, the focus shifts to the mistakes the system still make. Its robustness is put under pressure, and its results are compared with other datasets. Before moving on, the section pauses to take stock. Some things the approach does very well. Others still need work. The section weighs the two side by side. After that, Section 4.6 offers a short summary to close the chapter.

4.2 System Setup and Environment

All experiments were run on a single desktop workstation, with no cloud services or specialized accelerators. This section describes the hardware, the software environment and the reproducibility measures, and it reports how long the main experiments took.

4.2.1 Hardware

Table 4.1 lists the workstation's specifications. Every model in this thesis is a tree-based method that runs on the CPU. The learning libraries were configured to use all processor cores in parallel. No GPU acceleration was used, so the results can be reproduced on any comparable multi-core machine with enough memory.

Table 4.1 Hardware specification

Component	Specification
Processor	AMD Ryzen 7 7800X3D (8 cores and 16 threads)
Memory (RAM)	32 GB

Component	Specification
Graphics card	Nvidia GeForce RTX 3070 (not used for training)
Storage	1 TB Gen4 NVME
Operating system	Microsoft Windows 11, 64-bit

Memory, not processing power, was the main hardware constraint. The full release (3,540,241 flows) was loaded with single-precision features to roughly halve its memory footprint, and out-of-fold predictions, which take time to compute, were cached to disk so later experiments could reuse them without retraining.

4.2.2 Software Environment

The system was implemented in Python inside a dedicated Miniconda environment, used both as the Jupyter kernel for the notebooks and as the interpreter for the standalone experiment scripts (Table 3.10). Development was done in VS Code.

Two kinds of program were used. Jupyter notebooks were used for exploration and early-stage work, which follows the seven stages of the machine-learning workflow and shows the evidence for each decision before acting on it. Standalone Python scripts were used for the longer experiments on the full release and for the robustness studies. Each script is stored with the plain-text log and the JSON results file it produced, so every number in this thesis can be traced to the run that generated it.

4.2.3 Reproducibility Measures

- **Fixed random seed.** The value is 42 seeds for every split, fold, model and resampling step.
- **One environment.** All results were produced in the same Conda environment with fixed package versions.
- **Logged runs.** Every script writes a UTF-8 log and a machine-readable JSON file of its results. Reported numbers are copied from these files.
- **Cached intermediate results.** Out-of-fold predictions are saved as compressed NumPy archives, so experiments that change only the combiner, or only the test inputs, reuse identical member predictions.
- **Same-run controls.** When a component is compared with the existing pipeline, that pipeline is re-run in the same script. This removes run-to-run differences from parallel tree construction, which were measured at about ±0.001 macro-F1.

4.2.4 Execution Time

On the curated file, five-fold cross-fitting of the gate and the seven Stage-B members took 7 to 8 minutes, and the complete service-identity experiment, two full pipelines with bootstrap, took 19 minutes. On the full release, the v7 pipeline took about 15 minutes, refitting the gate and Stage-B pool on all training data took 3 to 6 minutes, and refitting the end-to-end combiner from cached predictions took 5.7 minutes.

Training cost is dominated by cross-fitting, which trains each of the eight models five times. Classifying a flow requires one gate evaluation, at most seven member evaluations and one weighted sum, which is far cheaper.

4.3 Implementation of the System

The system was implemented as a pipeline of independent modules (data loading, cleaning, splitting, feature building, cross-fitting, reject-set construction, combiner fitting, selection, final training, prediction and evaluation), each reading the output of the previous one and writing a well-defined result. This section describes the two workflows the implementation supports, training and prediction, and how the modules connect.

4.3.1 Training Workflow

The training workflow turns the raw dataset into a trained, validated system in the following order:

- **Load and verify.** The feature and label files are read and the label encoding is checked against the dataset's text labels.
- **Clean.** Exact duplicates are removed, then constant and highly correlated features are pruned.
- **Split.** The data is split into training (80%) and test (20%) sets, and the training set is further split into a combiner half and a selection half. All splits are stratified and seeded.
- **Build features.** The behavioral ratios and service-identity features are computed for every flow.
- **Cross-fit.** Over five folds of the training set, the gate and the seven members are trained on four folds and predict the fifth. The out-of-fold predictions are cached.
- **Build the reject set.** Benign training flows that the out-of-fold gate flagged as

attacks form the reject class, and the Stage-B members are cross-fitted on attack flows plus these rejects.

- **Fit the combiner.** On the combiner half, the weight matrix W is fitted by coordinate ascent on end-to-end macro-F1.
- **Select.** On the selection half, the candidate combiner variants are compared and the best is chosen.
- **Refit.** The gate and all members are retrained once on the full training set.
- **Evaluate.** The test set is scored once, metrics and confidence intervals are computed, and all results are logged.

4.3.2 Prediction Workflow

When a new flow arrives, the trained system processes it as follows:

- **Feature construction.** The flow's 76 statistics are reduced to the selected base features, and the 31 behavioral ratios and 26 service-identity features are computed from the flow's own values and its destination port and protocol.
- **Detection.** The gate estimates the probability that the flow is an attack. If it is 0.5 or below, the flow is reported as benign and processing ends.
- **Attribution.** Otherwise, each of the seven Stage-B members outputs a probability for each of the ten Stage-B classes (nine families plus reject).
- **Combination.** The probabilities are weighed by W and, if the selected variant uses it, by each member's accuracy on the flow's nearest neighbors in the reference set. They are then summed per class.
- **Decision.** If the highest-scoring class is a family, an alert with that family label is raised. If it is rejected, the flow is reported as benign.

The prediction path has no training-time dependencies other than the stored models, W and the neighbor reference set. Every step is a fixed computation per flow, so flows can be classified in batches or one at a time.

4.3.3 Integration and Verification

Not every error announces itself. A small mismatch can slip through unnoticed and quietly distort the results when the parts of a system are joined together. To catch problems like these, several automated checks were written into the implementation. Each is explained

below.

- **Disjoint partitions.** Assertions verify that training, combiner, selection and test indices do not overlap and that later experiments use the same splits as earlier ones.
- **Reproduction checks.** Before testing anything new, each experiment first goes back to the starting point. It re-runs the existing pipeline and checks that the score matches the one already published. A tiny difference is expected, since repeated runs wobble by about ±0.001. Only once the old result holds up does the new component get measured.
- **Equivalence checks.** When a new component generalizes an existing one, its degenerate setting is required to reproduce the old predictions exactly. For example, per-region weight matrices with a single region reproduced the global combiner on 100.0000% of test flows.
- **Physical consistency.** In the robustness simulation, every perturbed flow has all derived features recomputed, and a sanity check confirms that unperturbed features are unchanged.

4.4 Evaluation and Testing

4.4.1 Performance Metrics

Each prediction is compared with the true label. To see how these outcomes work, pick one class and call it *c*. Every flow the system labels can then land in one of a few places. If the flow really belongs to class *c* and the system calls it *c*, that's a hit, known as a true positive (TP). Sometimes the system calls a flow *c* when it actually belongs to a different class. That's a false alarm, or false positive (FP). The opposite can happen too. A flow that truly belongs to *c* gets labelled as something else, and this miss is called a false negative (FN). The following metrics are reported.

$$Accuracy = \frac{number\ of\ correctly\ classified\ rows}{total\ number\ of\ flows}$$

$$Precision_c = \frac{TP_c}{TP_c + FP_c}$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}$$

$$F1_c = \frac{Precision_c \cdot Recall_c}{PRecision_c + Recall_c}$$

$$MacroF1 = (1/K) \cdot \sum_c F1_c$$

$$WightedF1 = \sum_c n_c \ /N \cdot F1_c$$

Accuracy measures the fraction of all flows classified correctly. Because benign traffic dominates, accuracy is reported but never used to compare models. **Precision** for a class answers "when the system says this attack, how often is it right?", which relates to the false-alarm burden on analysts. **Recall** answers "of all attacks of this kind, how many did the system catch?", which relates to missed attacks. **F1-score** is the harmonic mean of the two. **Macro-averaged** precision, recall and F1 weight all classes equally, so a rare family such as Worms counts as much as Benign. **Weighted** averages weight classes by their size and therefore mostly reflect benign performance. **Macro-F1 is the primary metric** of this thesis because the research problem is precisely the detection of rare attack families.

ROC-AUC (area under the receiver operating characteristic curve) measures how well the predicted probabilities rank flows of a class above flows of other classes, independently of any decision threshold. It is computed one-vs-rest for each class and macro-averaged, from the models' probability outputs. **Confusion analysis** counts how many flows of each true class were assigned to each predicted class and identifies which families are confused with each other. For the deployed two-stage system, two further operational counts are reported: benign flows raised as attacks (false alarms) and real attacks withdrawn by the reject class.

4.4.2 Testing Approach

Hold-out testing. Each experiment uses a stratified 20% test set that is never used for any decision and is scored once (Section 3.6.6). Model and combiner choices are made on the selection half of the training data.

Bootstrap confidence intervals. The test set is a finite sample, so a score could differ on another sample of the same traffic. To put a number on that uncertainty, the test set was reshuffled again and again. Each round drew a fresh sample of flows from the original set, with replacement, while keeping every class in its true proportion. This was repeated 2,000 times, and the score was calculated anew each time. The result was a spread of 2,000 possible scores. Trimming off the lowest 2.5% and the highest 2.5% leaves the middle range, and that range is the 95% confidence interval (Efron & Tibshirani, 1993). A single configuration's macro-F1 interval is roughly ±0.017.

Paired significance tests. To decide whether configuration B improves on A, both were scored on the same resample in every iteration, and the 2,000 paired differences give a confidence interval and a p-value for the improvement. Pairing removes the variation that both configurations share and makes the test far more sensitive than comparing two separate intervals. The paired interval was computed for each specific comparison, because its width depends on how much the two pipelines share. It ranged from about ±0.0025 for pipelines differing in one component to about ±0.012 for pipelines differing in many. **An improvement is established only when its paired interval excludes zero.**

Ablation testing. Each component was tested by removing or replacing it while keeping everything else fixed (same data, splits, seeds and protocol). This attributes performance changes to individual design decisions.

Robustness testing. The trained model was evaluated on perturbed copies of the test attack flows at increasing perturbation budgets, under random-noise, evasion and misattribution threat models (Section 3.6.6). When the robustness of two models is compared, their decision thresholds are first set to the same false-positive rate, because otherwise a model that simply raises fewer alerts can look more robust.

Explainability evaluation was outside the scope of this thesis. Explainability methods such as SHAP (Lundberg & Lee, 2017) were not applied, and Section 6.4 identifies this as future work.

4.5 Results and Discussion

The section describes the findings of the result step by step. It started with three standard classifiers. With the works this section also shows the ideas that did not work. Every score comes from a test set that was never used during development which were used only once. Based on 2,000 bootstrap resamples of the test set, each headline value has a 95% confidence interval. (Efron & Tibshirani, 1993). Results on the curated file use 61,235 test flows. Results on the full release use 521,361.

4.5.1 Baseline Classifiers

We started with three familiar models. A single decision tree (Breiman et al., 1984) sets the floor. What stronger learners can do was demonstrated by a random forest of 200 trees (Breiman, 2001) and XGBoost with 400 boosting rounds (Chen & Guestrin, 2016). All the three models used balanced class weights and the same 44 features.

Table 4.2 Baseline classifiers with and without supervised feature selection, curated test set, 61,235 flows

Model	Accuracy	Macro P	Macro R	Macro-F1	Weighted F1	ROC-AUC
Decision tree, 44 features	0.8855	0.5408	0.5284	0.5338	0.8856	0.7557
Decision tree, 36 features	0.8847	0.5361	0.5282	0.5316	0.8849	0.7556
Random forest, 44 features	0.9017	0.6521	0.5358	0.5744	0.9004	0.9401
Random forest, 36 features	0.9024	0.6580	0.5404	0.5805	0.9012	0.9428
XGBoost, 44 features	0.8949	0.5422	0.6510	0.5807	0.9021	0.9803
XGBoost, 36 features	0.8942	0.5402	0.6534	0.5800	0.9016	0.9803

Table 4.2 shows the gap that helped with the further decisions. Weighted F1 was between 0.8849 and 0.9021 for every model. But the macro-F1 between 0.5316 and 0.5807 has shown a different picture. The models identified the benign traffic and large attack families well, but they struggled to detect the rare attack families. Swapping the single tree for XGBoost increased the macro-F1 by only 0.047, from 0.534 to 0.581. From this, we got the hint that a bigger model is not the answer. Trimming the features from 44 to 36 with mutual information and recursive elimination made no real difference either.

The two ensembles also fail in opposite ways. Random forest plays it safe, with high precision and low recall. XGBoost is bolder. It catches more rare attacks but raises more false alarms on the way.

4.5.2 Class-Imbalance Handling

That is what full SMOTE oversampling (Chawla et al., 2002) does, balancing every class up to the size of the largest one, and many recent detectors lean on SMOTE or one of its variants (Almuhanna & Dardouri, 2025; Dasari et al., 2026; Nassreddine et al., 2025). Here, though, it backfired. XGBoost's macro-F1 dropped from 0.5807 to 0.5088. Precision and recall fell together. Worms show why. It started with only about 190 real training flows and ended up with around 180,000 synthetic ones, which is a lot to build from so little. Most of the new points were noise. Worms precision collapsed to 0.06.

Weighting the loss turned out to be a gentler and better tool. The winner was a tempered weight with α set to 0.25. The familiar 'balanced' setting, which is α equal to 1, ranked only twelfth. It even finished below using no weights at all. A second, independent family of weights peaked at a matching point. Both agree that the rarest family should weigh roughly

3 to 11 times the largest, where 'balanced' demands 128 times. Focal weighting, which leans on the hardest examples, went the other way and collapsed at its strongest setting. The lesson is simple. Moderate rebalancing works. Full rebalancing does not.

Table 4.3 Imbalance treatments compared on the test set, curated test set, 61,235 flows

Configuration	Selected on	Macro-F1	Macro P	Macro R
Flat XGBoost	**not applicable**	**0.5807**	**0.5422**	**0.6510**
Flat XGBoost + full SMOTE	**not applicable**	**0.5088**	**—**	**—**
Two-stage + balanced weights	**not applicable**	**0.5858**	**0.5427**	**0.6779**
Two-stage + α = 0.25	**validation split**	**0.6157**	**0.6574**	**0.6035**

Capped SMOTE was also tested fairly on the validation split, where it scored 0.5988 against 0.6123 for α = 0.25.

Table 4.3 shows the same pattern on the test set. Tempered weighting comes out at 0.6157, ahead of both the flat model and the two-stage system with balanced weights. Fuzzers became a false-positive sink, with precision 0.589 against recall 0.902, absorbing 1,116 benign flows that passed the gate.

4.5.3 Two-Stage Architecture and Ensemble

Next came a different idea, which was to split the problem in two. First a gate decides whether a flow is an attack at all. Then a second stage names the family. The gate? That turned out to be the easy part. On the curated data it caught about 99.7% of attacks, at roughly 92% precision. Naming the family was where things got stuck.

The split, on its own, didn’t help much. With balanced weights the two-stage system scored 0.5858, which is almost exactly where flat XGBoost already was. Pair it with tempered weighting, though, and it added a significant 0.018 (see Section 4.5.6). So the gain came from the two together, not from the hierarchy by itself.

Table 4.4 Ensemble designs on the 44 base features, curated test set, 61,235 flows

Configuration	Accuracy	Macro P	Macro R	Macro-F1	Weighted F1
Algorithm-diverse dynamic ensemble	0.905	0.661	0.593	0.612	0.906
α-diverse ensemble with per-class weights	0.903	0.673	0.633	0.630	0.906
Cross-fitted α-diverse ensemble	0.906	0.680	0.642	0.633	0.910

Ensembles came next, and Table 4.4 sets out the designs that were tried. The first attempt mixed different algorithms, namely XGBoost, a random forest and extra-trees, and chose between them flow by flow, in the spirit of dynamic classifier selection (Cruz et al., 2018; Woods et al., 1997). It added nothing. Once weighting was tempered, the three models tended to make the same mistakes, so there was little for a combiner to work with.

The breakthrough came from varying the members along the weighting axis instead. The pool held XGBoost models with α set to 0, 0.25, 0.5 and 1, a capped-SMOTE model, a random forest and an extra-trees model. The member with α equal to 1 was one of the weakest on its own. Yet it disagreed with the others on 15 to 23% of flows, and that disagreement was useful. A learned weight for every member and class turned the pool into a gain. A stacked meta-classifier (Wolpert, 1992) with the same number of parameters overfitted badly by comparison. Cross-fitting the pool over all the training data lifted the score again, to 0.6339.

Per-class decision thresholds and gate re-weighting were also tested and set aside (Table 4.13).

4.5.4 Engineered and Service-Identity Features

By this point the models had plateaued around 0.63, so attention turned from the model to the data it sees. Thirty-one engineered ratio features were added first. They describe how a flow behaves rather than how big it is. Together with the ensemble, they lifted macro-F1 to 0.649 [0.631, 0.666]. The share of packets carrying the PSH flag became the second most important feature in the whole model.

The biggest jump in the whole project came from one missing piece of information. Which service was each flow aimed at? The curated file simply leaves out the destination port and protocol. So they were recovered from the full release and turned into 26 service features. The payoff was large, a paired gain of 0.098 macro-F1, 95% CI [0.089, 0.108], with p

below 0.001. That's roughly twice what the entire modelling stack had managed before it. On the curated data, the system now reached 0.746 [0.727, 0.763]. Gate precision rose from 0.9267 to 0.9411 at unchanged recall (0.9964).

Table 4.5 Per-class F1 change from service-identity features, same-run paired comparison

Family	F1 without service	F1 with service	Change
Benign	0.9849	0.9880	+0.0031
Analysis	0.4312	0.4519	+0.0207
Backdoor	0.6351	0.7034	+0.0683
DoS	0.4643	0.5601	+0.0958
Exploits	0.7916	0.8499	+0.0583
Fuzzers	0.7232	0.8989	+0.1757
Generic	0.8296	0.8842	+0.0546
Reconnaissance	0.6424	0.8327	+0.1903
Shellcode	0.3759	0.6579	+0.2820
Worms	0.6000	0.6329	+0.0329

Table 4.6 Top 15 Stage-B features by importance with service identity

Rank	Feature	Importance	Type
1	svc_rpc	0.3721	service
2	svc_bgp	0.0606	service
3	frac_PSH	0.0435	engineered
4	SYN Flag Count	0.0349	base
5	svc_ambiguous	0.0291	service
6	svc_smb	0.0273	service
7	svc_pptp	0.0267	service
8	svc_http	0.0235	service
9	svc_rip	0.0172	service
10	PSH Flag Count	0.0172	base
11	dst_port_num	0.0165	service
12	svc_other_low	0.0163	service
13	Bwd Packet Length Max	0.0149	base
14	svc_registered	0.0149	service
15	svc_imap	0.0145	service

Service columns make up 25.7% of the features but carry 70.4% of the XGBoost gain importance.

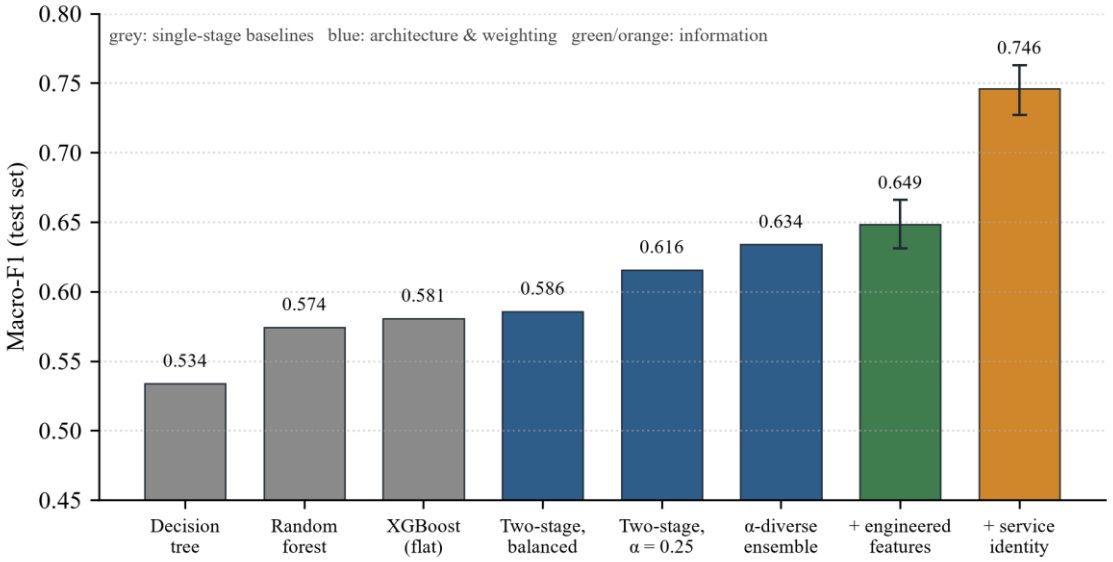


Figure 4.1 Macro-F1 progression on the curated test set, with 95% bootstrap confidence intervals where they were computed

4.5.5 Final System at the Realistic Base Rate

Truth is, the curated file makes things look easier than they really are. It has about ten times fewer benign flows than the real capture did. So how does the system hold up in real conditions? To find out, it was retrained and tested on the full release. One warning, though. The mix of classes is very different here, which means these results can't be lined up directly against the curated ones.

Table 4.7 Full-release class distribution after deduplication

Label	Class	Flows	Share (%)
0	Benign	2,525,314	96.8740
1	Analysis	385	0.0148
2	Backdoor	452	0.0173
3	DoS	4,418	0.1695
4	Exploits	30,925	1.1863
5	Fuzzers	26,433	1.0140
6	Generic	4,496	0.1725
7	Reconnaissance	12,035	0.4617
8	Shellcode	2,102	0.0806
9	Worms	243	0.0093
	Total	2,606,803	100.00

Deduplication reduced 3,540,241 flows to 2,606,803. The stratified 80/20 split gave 2,085,442 training flows and 521,361 test flows.

Table 4.7 shows how lopsided the real traffic is. Benign flows make up almost 97% of the data, and Worms has just 243 flows left after deduplication.

The first full-release version, v7, kept its grip on telling attacks apart. The trouble lay with the families that look like normal traffic. **8,370 benign flows slipped past the gate, and most of them, 6,327 in fact, ended up labelled as Fuzzers. No surprise, then, that Fuzzers precision dropped to 0.44.**

The fix was to let the second stage say no. A reject class, trained on benign flows that had fooled the gate during cross-validation, sends such flows back as benign. Version v7.1 added it.

For the final system, the combiner weights were refitted to score the whole pipeline at once, and the gate threshold was chosen on held-out data. On the selection split, gate thresholds from 0.3 to 0.7 all scored within 0.0034 macro-F1 of each other (0.6700 to 0.6734), so the threshold barely matters once the reject class is in place.

Table 4.8 Final system, overall test metrics, full-release test set, 521,361 flows

Metric	Weighted	Macro
Precision	0.9880	0.7117
Recall	0.9866	0.6895
F1-score	0.9872	0.6887
Accuracy	0.9866	
ROC-AUC, one-vs-rest		0.9644

Hybrid per-class weights with local-accuracy weighting, k = 30 and p = 3, gate threshold 0.7. Macro-F1 95% CI [0.672, 0.704].

Table 4.8 gives the final result, Table 4.9 breaks it down by class and Figure 4.2 shows the full confusion matrix. The system reaches a macro-F1 of 0.689 [0.672, 0.704], an accuracy of 98.66% and a macro ROC-AUC of 0.964. Benign, Exploits, Generic and Reconnaissance are recognized reliably. Analysis is the clear failure. Most of its few test flows look so much like normal traffic that they are returned as benign: Figure 4.2 shows 47 of its 77 flows in the Benign column.

Table 4.9 Final system, per-class results, full-release test set, 521,361 flows

Class	Precision	Recall	F1-score	Support
Benign	0.9962	0.9935	0.9949	505,063
Analysis	0.1053	0.3377	0.1605	77
Backdoor	0.8065	0.5556	0.6579	90
DoS	0.6592	0.5645	0.6082	884
Exploits	0.8049	0.8427	0.8234	6,185
Fuzzers	0.5702	0.7282	0.6396	5,287
Generic	0.8788	0.8788	0.8788	899

Class	Precision	Recall	F1-score	Support
Reconnaissance	0.8809	0.7740	0.8240	2,407
Shellcode	0.8571	0.6286	0.7253	420
Worms	0.5577	0.5918	0.5743	49

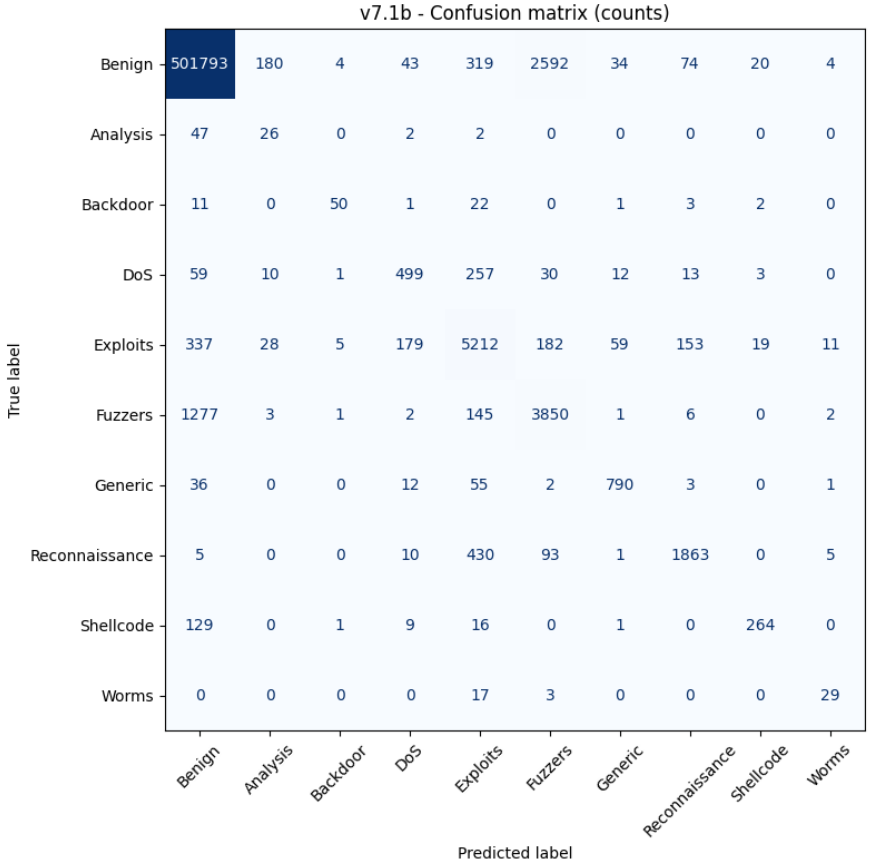


Figure 4.2 Confusion matrix of the final system, full-release test set, 521,361 flows

Flows sent to the reject class count as Benign. The Benign row holds the false alarms and the Benign column holds attacks returned as benign. Rows show the true class and columns the predicted class. Correct predictions sit on the diagonal.

Table 4.10 Reject-class accounting for the final system, full-release test set, 521,361 flows

Quantity	Value
Gate attack recall at threshold 0.7	0.9772
Gate attack precision at threshold 0.7	0.6833
Benign flows raised as attacks, final system	3,270
Benign flows raised as attacks, v7 without reject class	8,370
Gate-positive flows returned as Benign	5,643
Of these, genuinely benign	4,113
Of these, real attacks lost	1,530
Alerts raised	17,667
Genuine alerts	14,397
Share of alerts that are false	19%

The reject class is a trade, not a free win, and Table 4.10 lays out both sides. Benign false

alarms fell from 8,370 to 3,270. The price was 1,530 real attacks that were sent back as benign along with them. Most of these were Fuzzers flows, because Fuzzers and benign traffic overlap so closely. Someone has to pay, either in missed attacks or in false alarms. Deciding which is a judgement call for the security team. Even so, about one alert in five is still false at this base rate.

Table 4.11 Improvement ladder on the full release with paired significance, all configurations from one run, full-release test set, 521,361 flows

Configuration	Macro-F1	Macro P	Macro R	Accuracy	Paired change in macro-F1 [95% CI]	p
v7, no reject class	0.6584	0.6384	0.7533	0.9798	—	—
v7.1, with reject class	0.6848	0.7137	0.6833	0.9873	+0.0265 [+0.0172, +0.0359]	< 0.001
v7.1b, with end-to-end objective	0.6884	0.7095	0.6963	0.9865	+0.0036 [+0.0020, +0.0056]	< 0.001
Final, chosen on the selection split	0.6887	0.7117	0.6895	0.9866	+0.0304 [+0.0211, +0.0404] vs v7	< 0.001

Changes are paired bootstrap differences (2,000 stratified resamples) against the previous row; the final row is compared with v7.

Table 4.11 puts numbers on each step. Adding the reject class raised macro-F1 by 0.027. Refitting the combiner for the whole pipeline added another 0.004. Both gains are statistically established. Shellcode gained the most, rising from 0.504 to 0.725, while DoS and Fuzzers each gained about 0.04. The reject class traded Fuzzers recall, which fell from 0.94 to 0.73, for precision.

4.5.6 Ablation and Statistical Significance

Table 4.12 Paired significance of each component, 2,000 bootstrap resamples

Component	Change in macro-F1	95% CI	p	Established
Tempered weighting, α = 0.25, two-stage	+0.0385	[+0.0246, +0.0512]	< 0.001	Yes
Two-stage hierarchy at α = 0.25	+0.0178	[+0.0061, +0.0311]	0.005	Yes
α-diverse ensemble	+0.0177	[+0.0067, +0.0301]	0.003	Yes
Tempered weighting, flat model	+0.0146	[+0.0001, +0.0276]	0.049	Marginal
Two-stage hierarchy at balanced weights	−0.0061	[−0.0144, +0.0030]	0.174	No
Full stack versus flat XGBoost	+0.0502	[+0.0391, +0.0606]	< 0.001	Yes
Service-identity features	+0.0982	[+0.0885, +0.1081]	< 0.001	Yes

Component	Change in macro-F1	95% CI	p	Established
Reject class, full release	+0.0265	[+0.0172, +0.0359]	< 0.001	Yes
End-to-end combiner objective	+0.0036	[+0.0020, +0.0056]	< 0.001	Yes
Hyperparameter tuning, 40 configurations	+0.0054	[−0.0037, +0.0136]	≈ 0.23	No
Reject-set enrichment	+0.0009	[−0.0082, +0.0096]	≈ 0.82	No
Per-region weight matrices	+0.0018	[−0.0034, +0.0076]	≈ 0.55	No
Feature-view member pool	+0.0066	[−0.0017, +0.0144]	≈ 0.11	No

Every component was tested by comparing two pipelines that differed only in that one part. Table 4.12 brings the results together. Only some changes made a real difference, and they fall into three groups. Weighting came first, adding 0.039, more than twice what the whole ensemble contributed. Information came next, with service identity delivering the biggest jump of all. Objectives completed the set, through the reject class and the end-to-end weights. The levers that only made the model bigger or more elaborate did not pay off. Tuning hyperparameters, widening the pool, adding per-region combiners and choosing members flow by flow all fell inside the noise.

Table 4.13 Approaches that did not improve the system

Approach	Outcome	Reason identified
Full SMOTE oversampling	0.5807 to 0.5088	Synthetic noise from oversampling tiny classes
Focal weighting	Collapse at $\gamma = 2$, 0.3497	Hard examples are ambiguous rather than informative
Algorithm-diverse dynamic ensemble	No gain on test	Members not complementary once weighting is tempered
Per-class decision thresholds	Selection gain reversed on test	Duplicates the per-class weight matrix
Gate threshold and gate re-weighting	+0.0006	Gate already well calibrated
Extending α beyond 1.0	0.6329 versus 0.6339	Members above $\alpha = 1$ are individually degraded
38 further algebraic features	−0.0021	Mostly rewrites of existing features
Dynamic versus static combination	+0.0026, not established	Local accuracy optimises error rate, not macro-F1

4.5.7 Analysis of the Remaining Errors

Table 4.14 The twelve most confused family pairs, comparing class overlap with exact label ambiguity

Family pair	k-NN overlap	Identical records with conflicting labels	Model confusions
Exploits and Fuzzers	0.3051	46	1287
Fuzzers and Reconnaissance	0.3372	0	818
Exploits and Reconnaissance	0.2466	0	451
DoS and Exploits	0.4932	0	442
Fuzzers and Shellcode	0.4935	134	282
DoS and Fuzzers	0.2324	47	245
Fuzzers and Generic	0.2085	87	180
Exploits and Generic	0.3902	21	92
Exploits and Shellcode	0.1908	0	77
Analysis and Exploits	0.5213	0	72
DoS and Reconnaissance	0.0960	22	43
Reconnaissance and Shellcode	0.1688	12	34

Why did progress level off before service identity arrived? Two explanations were tested. The first was noisy labels, and it did not hold up. Only 0.35% of flows share an identical feature vector with a flow of a different label, so a perfect classifier could in principle score far higher. The second was class overlap, and this one fitted. Many families simply sit on top of each other in the flow features. Table 4.14 shows that this overlap tracks the model's confusions closely, with a Spearman correlation of 0.827. It even explains pairs that exact matching cannot. Exploits and Reconnaissance, for instance, were confused 451 times without a single identical record between them. Exploits and Fuzzers act like magnets. Over half of an Analysis flow's nearest neighbours are Exploits flows. This geometry explains why bigger models did not help, and why information that separates look-alike families helped so much.

4.5.8 Robustness to Perturbed Inputs

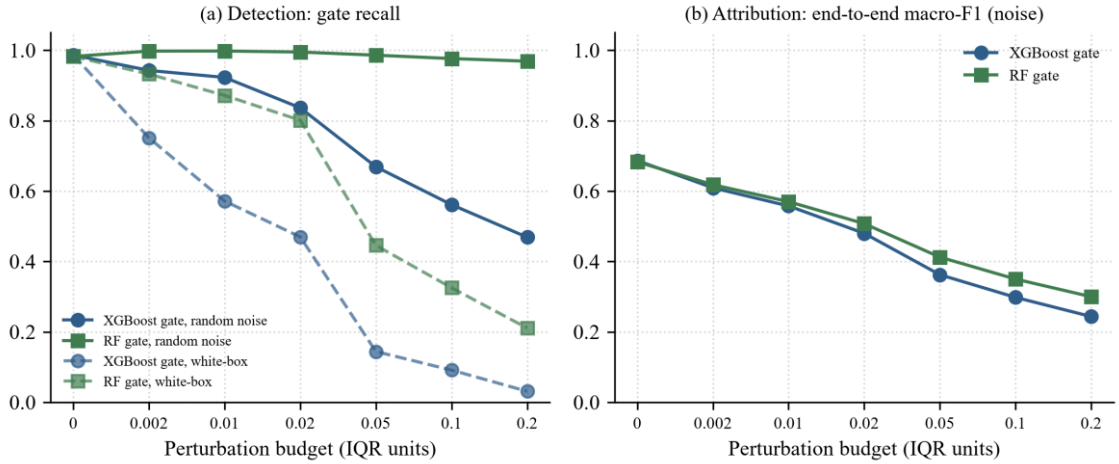


Figure 4.3 Robustness of detection, panel a, and attribution, panel b, as the perturbation budget grows

A detector also has to survive messy or hostile input. To test this, the test attacks were nudged by growing amounts, measured in units of each feature's interquartile range. Every dependent feature was recomputed after each change, so the altered flows still made physical sense, as Apruzzese et al. (2022) recommend. Figure 4.3 summarises what happened.

Even random noise hurt, as Table 4.15 shows. A nudge of just 0.002 cut macro-F1 from 0.687 to 0.609. At the largest budget, the XGBoost gate caught only 47% of attacks. The gate proved to be the lever. Swapping in a random-forest gate kept gate recall at 0.97 under the same noise, at a small cost of 0.004 on clean data.

A deliberate attacker is another matter. Earlier adversarial work, such as Tafreshian and Zhang (2024), focuses on evasion, where malicious traffic is disguised as benign. In the white-box test (Table 4.15), evasion drove XGBoost gate recall down to 3%. The random-forest gate held on better, at 21%, so it is more robust but far from safe. An attack built against XGBoost and reused on the random forest left it catching 88% of attacks. Testing with a stand-in model, in other words, makes a defence look much stronger than it is.

Table 4.15 Robustness of the deployed system under random noise and targeted evasion, full-release test set, 521,361 flows

Threat model	Gate	Stage B	Macro-F1 at budget 0	0.002	0.02	0.2	Gate recall at budget 0	0.002	0.02	0.2
Random noise	XGBoost	XGBoost (deployed)	0.6865	0.6089	0.4797	0.2433	0.9862	0.9425	0.8363	0.4686
Random noise	Random forest	XGBoost	0.6827	0.6178	0.5072	0.2996	0.9823	0.9975	0.9948	0.9688

Threat model	Gate	Stage B	Macro-F1 at budget 0	0.002	0.02	0.2	Gate recall at budget 0	0.002	0.02	0.2
Random noise	XGBoost	Random forest	0.6617	0.6120	0.4619	0.2377	0.9862	0.9425	0.8363	0.4686
Random noise	Random forest	Random forest	0.6563	0.6208	0.4855	0.2953	0.9823	0.9975	0.9948	0.9688
White-box evasion	XGBoost	XGBoost	0.6865	0.5494	0.3734	0.1446	0.9862	0.7508	0.4695	0.0309
White-box evasion	Random forest	XGBoost	0.6827	0.5795	0.4608	0.2146	0.9823	0.9321	0.8010	0.2105
Evasion crafted on XGBoost, applied to RF	Random forest	XGBoost	0.6827	0.5987	0.4823	0.3000	0.9823	0.9921	0.9705	0.8758

Budgets are in units of each feature's interquartile range. Random-forest thresholds are matched to the XGBoost false-positive rate.

The study also tried a new kind of attacker, one who does not hide at all. This misattribution attacker lets the alert fire but changes the family it reports. Gate recall stayed near 0.997, yet 63% of attack labels came out wrong. It needed about 3.8 times more perturbation than evasion. Its strength is stealth, because alert volumes look perfectly normal. It also turned the reject class to its advantage, pushing abstentions from 9% to 22% at the smallest budget. Table 4.16 compares the outcome rates under the two objectives.

Table 4.16 Outcome rates under the evasion and misattribution objectives, full-release test set, 521,361 flows

Objective	Budget	Undetected	Misattributed	Abstained	Gate recall	Macro-F1
Clean	0	0.1017	0.1122	0.0879	0.9862	0.6865
Evasion	0.002	0.2667	0.1112	0.0175	0.7508	0.5494
Evasion	0.02	0.5576	0.0712	0.0271	0.4695	0.3734
Evasion	0.2	0.9697	0.0038	0.0006	0.0309	0.1446
Misattribution	0.002	0.2282	0.1887	0.2238	0.9955	0.4712
Misattribution	0.02	0.2811	0.3188	0.2784	0.9974	0.2917
Misattribution	0.2	0.2571	0.6311	0.2541	0.9969	0.1348

4.5.9 Results on Other Datasets

Table 4.17 External checks on other datasets

Dataset or experiment	Macro-F1	Accuracy	Key observation
Original UNSW-NB15, same traffic with a different extractor	0.6387	0.8303	Service fields carry most of the predictive power
CIC-IDS2017, 13 classes	0.9835	0.9991	A 1-NN rule alone is 99.7% accurate

Dataset or experiment	Macro-F1	Accuracy	Key observation
TON_IoT audit	—	—	Source IP alone gives 0.9998 attack recall

To see whether the findings held up beyond one dataset, the pipeline was run on three others, summarised in Table 4.17. The original UNSW-NB15 release (Moustafa & Slay, 2015) came first. Same traffic, different tool turning it into features. Here the system scored 0.639, and, once again, the service fields did most of the work, backing up the service-identity result from a separate source. CIC-IDS2017 (Sharafaldin et al., 2018) looked very different on the surface, with a macro-F1 of 0.984. A closer look shows that the credit belongs to the dataset.. Even a one-nearest-neighbour rule got 99.7% of flows right. Every combiner variant finished within 0.0007 of the others, too. TON_IoT (Moustafa, 2021) was similarly easy. Its source IP address alone gave the attacks away, and that reflects how the testbed was built, not how attacks behave.

Duplicate records are a known trap in intrusion datasets (Tavallaee et al., 2009). One last experiment deliberately left them in. Macro-F1 barely moved, from 0.6378 to 0.6403, and accuracy actually fell from 0.8299 to 0.8228, even though 42.7% of test flows then had an exact copy in training. The flows with copies actually scored worse, 0.8064 against 0.8350, so the model was not simply memorising. The duplicates did badly distort the nearest-neighbour diagnostics, though, cutting the 1-NN error from 0.3044 to 0.2435, which is why deduplication stays in the pipeline.

4.5.10 Comparison with Existing Approaches and Discussion

Table 4.18 This work alongside the reviewed studies

Study	Approach	Imbalance handling	Data	Reported result
Thockchom et al. (2023)	Stacked naive Bayes, logistic regression and decision tree, chi-square selection	Not the main focus	KDD Cup 1999, UNSW-NB15, CIC-IDS2017	Ensemble beats its base classifiers
Dasari et al. (2026)	Stacked extra-trees and XGBoost, Boruta selection	K-means SMOTE	Not given in abstract	Accuracy 97.85%, F1 97.82%
Nassreddine et al. (2025)	Stacked RF, DT, k-NN and XGBoost, correlation plus embedded selection	SMOTE-Tomek	NSL-KDD, CIC-IDS	Accuracy 99.99%
Emirmahmutoğlu and Atay (2025)	Heuristic feature selection with ten classifiers	Not the main focus	KDD Cup 1999, NSL-KDD, UNSW-NB15, CSE-CIC-IDS2018	Best UNSW-NB15 F1 0.9507
Farhan et al. (2025)	Deep neural network, extra-trees selection	Not the main focus	UNSW-NB15	Binary accuracy 97.93%

Study	Approach	Imbalance handling	Data	Reported result
Almuhanna and Dardouri (2025)	Weighted soft-voting ML and DL ensemble	SMOTE	5.6 million records	Near 100% on all metrics
Uppal et al. (2025)	Max-voting ensemble with XGBoost	Not the main focus	KDD Cup 1999	Accuracy 99.82%
Uddamari and Sammulal (2025)	Adaptive dynamic ensemble with drift detection	Adaptive weight rebalancing	CIC-IDS2017, UNSW-NB15	Accuracy 98.1% and 96.8%
Tafreshian and Zhang (2024)	Adversarial training with ensemble learning	Dataset balancing	NSL-KDD, UNSW-NB15	35% higher accuracy under attack
This work	Two-stage α-diverse ensemble with reject class	Tempered weights, α = 0.25	CIC-UNSW-NB15, curated and full release	Macro-F1 0.746 curated, 0.689 full release, accuracy 98.66%

Results of the reviewed studies are as stated in their abstracts. They use different datasets, tasks and protocols, so the figures are not directly comparable.

Table 4.18 places this work next to the studies reviewed in Chapter 2. Lining up the numbers would be tempting, but it would not be fair. Most of the reviewed studies report accuracy or binary results, often on older benchmarks, and as the survey by Ahmad et al. (2021) shows, protocols vary widely across the field. None of the ten reviewed papers released its code, so their evaluation choices cannot be checked. Such choices include whether duplicates were removed before splitting, which is a well-known source of inflated scores (Arp et al., 2022; Kaufman et al., 2012).

This study's own numbers show why accuracy can mislead. The final system reaches 98.66% accuracy, well within the range other papers report. Yet its macro-F1 of 0.689 reveals that Analysis and Worms are still poorly detected. The 0.984 macro-F1 on CIC-IDS2017 makes the same point from the other side, since it came from an easy dataset rather than a better model. This work is therefore best judged by its controlled, statistically tested gains over its own baselines, and by how openly its protocol is set out.

Some things the approach does very well. The evaluation rules out leakage, every component was tested on its own, and every number carries a confidence interval. The system improves a great deal on standard baselines in the area that matters most, the rare attack families. It was tested at a realistic base rate, and its failures can be explained, not just reported. Other things still need work. Analysis reaches an F1 of only 0.16, and Worms rests on just 49 test flows. False alarms gather along the Fuzzers boundary. Part of the service-identity gain may be specific to this testbed. Small input changes can throw the model off, and a targeted attacker can tamper with its labels. Every result also comes from a single testbed.

4.6 Summary

This chapter followed the system from setup to results. It was built as a modular pipeline designed to prevent leakage. On the curated dataset, macro-F1 started at 0.581 with XGBoost. Tempered weighting, the two-stage hierarchy, the ensemble and the engineered features lifted it to 0.649. Service identity then pushed it to 0.746. At the realistic base rate, where **almost 97%** of traffic is benign, the final system reached a macro-F1 of 0.689 [0.672, 0.704] and 98.7% accuracy. Benign false alarms fell from **8,370 to 3,270**, about 2.6 times fewer with the reject class. The paired tests showed where the gains really came from. They came from moderate rebalancing, richer information and well-chosen objectives, not from a bigger model. The error analysis traced the remaining mistakes to classes that overlap in the flow representation. The robustness study found the system fragile under perturbation, though it also found a practical way to protect detection against random noise. Together, these findings support the research objectives while making the limits plain. The chapters that follow set them against standards, constraints and future work.

Chapter 5

Standards, Constraints and Milestones

5.1 Standards and Compliance

Intrusion detection touches three different worlds. It belongs to everyday security practice, it depends on software quality, and it now falls under the new rules taking shape for artificial intelligence. Several standards and frameworks apply to this work for this reason, and Table 5.1 brings them together.

Security practice is the natural starting point. NIST Special Publication 800-94 (Scarfone & Mell, 2007) explains what is a intrusion detection and prevention systems. It also describes how their performance should be assessed. By its definitions, the proposed system is a network-based IDS. Its evaluation also follows the guide's priorities, which are detecting attacks accurately and keeping the burden of false alarms low. The NIST Cybersecurity Framework 2.0 (National Institute of Standards and Technology [NIST], 2024) states continuous monitoring and the analysis of adverse events as its Detect function. This system was designed to maintain that standard. ISO/IEC 27001:2022 (International Organization for Standardization [ISO], 2022) matters here too. It asks organizations to keep an eye on their networks and catch anything that looks off. That's precisely the kind of job a flow-based NIDS is good at.

Machine learning, though, comes with worries of its own. Two frameworks deal with them head on. The first is the NIST AI Risk Management Framework (NIST, 2023). The second is ISO/IEC 23894:2023 (ISO, 2023b). Both make the same basic point. If you use AI, you need to write down its risks, measure them, and keep them under control. This thesis answers that call in four ways. Each score it reports carries a confidence interval. Ablation studies reveal what every component adds. A robustness study shows how the system behaves under pressure. Its limitations are also laid out in the open. Software quality was not forgotten either. The ISO/IEC 25010 product quality model (ISO, 2023a) guided the expectations here, and it fed straight into the non-functional requirements set out in Section 3.3.

Table 5.1 Standards and frameworks relevant to this work

Standard / framework	Relevance	How this work aligns
NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems	Defines IDS types, components and evaluation criteria	Network-based detector; evaluated on detection and false-alarm rates
NIST Cybersecurity Framework 2.0, Detect function	Continuous monitoring and adverse-event analysis	Provides per-flow detection and attack-family attribution
ISO/IEC 27001:2022	Requires network monitoring for anomalous behaviour	Can serve as a monitoring control within an ISMS
NIST AI Risk Management Framework 1.0	Map, measure, manage and govern AI risks	Uncertainty quantified, components ablated, robustness measured, limits documented
ISO/IEC 23894:2023, AI risk management	Guidance on risk management for AI systems	Risks of false alarms, missed attacks and adversarial manipulation identified
ISO/IEC 25010 product quality model	Quality characteristics for software	Basis for non-functional requirements (reliability, efficiency, maintainability)

The main limitation in compliance is that this is a research prototype. It has not been deployed in an operational environment, audited against these standards or integrated with an organisation's incident-response processes, all of which formal compliance would require.

5.2 Sustainability Considerations

The proposed system has been designed to be computationally efficient. It uses tree-based ensembles rather than deep neural networks. Tree ensembles train on a standard CPU in minutes (Section 4.2) and need no GPU, whose energy use for large model training has been a growing concern (Strubell et al., 2019). Inference is cheap: a flow is classified with one gate evaluation and at most seven member evaluations, and benign flows, the large majority of traffic, stop after the gate. Waste was something the research tried hard to avoid. Out-of-fold predictions, for instance, were saved after they were first made. Later experiments could simply load them instead of retraining every model from scratch. Many experiments went a step further and shared the very same trained models. Scale turned out to be less of a problem than expected. The same machine handled both 0.45 million and 3.5 million flows. Memory, not processing power, was what set the limit. Training cost also behaves predictably. As the number of flows grows, it rises roughly in step. The main sustainability cost of the approach is cross-fitting, which trains every model five times. In a production setting, models would be retrained only periodically, and this cost would be

spread over a long period of use.

5.3 Societal Impact

Good intrusion detection helps keep the services running that people depends on. It also gives organizations a chance to catch a breach early, before personal data is stolen or systems go down. This system adds one more thing. It does not just flag that an attack is happening. It also identifies the kind of attack. By knowing the optimal actions path, analysts can react faster. Its transparent and reproducible procedures allow the scientific community to validate and build upon the findings. There are potential negative impacts as well. False alarms consume analysts' time and can cause alert fatigue, so real attacks are eventually ignored. Relying too much on an automated detector can lead to a false sense of security, particularly since this thesis demonstrates that a determined opponent can trick or circumvent the system. Monitoring technologies can potentially be misused to gather information on authorized users. Knowing the vulnerabilities can help both attackers and defenders. When the system is utilized as decision support for qualified analysts, under proper governance, and with its limitations well disclosed, the advantages generally exceed the hazards.

5.4 Ethical Considerations

Several ethical issues were considered. Data privacy: all datasets used are public research datasets generated in controlled testbeds (UNSW-NB15 traffic was generated with the IXIA Perfect Storm tool in a cyber-range laboratory), so no real users' communications were processed and no personal data was collected. Bias and fairness: in this context bias means unequal performance across attack families rather than across people. The thesis addresses it directly by optimizing and reporting macro-F1, so that rare families count as much as common ones, and by reporting per-class results with confidence intervals so that weak families are not hidden in an average. A further form of bias was identified and avoided. Source IP addresses and TTL values identify the testbed machines rather than malicious behavior, so they were deliberately excluded from the model to prevent it from learning shortcuts. Honest reporting: negative results are reported alongside positive ones, numbers are quoted with intervals, and test-set selection was avoided, to prevent misleading readers about the system's capability. Responsible use: the system is intended for defensive monitoring of networks by their owners. The adversarial experiments were conducted

purely in simulation on recorded data, to measure and improve robustness, and no attacks were carried out against real systems.

5.5 Security and Privacy Considerations

A detector that protects a network also needs to be protected because it becomes a target in and of itself. There are four primary hazards that come to mind.

The first is manipulation by an attacker. Section 4.5.8 showed how small, realistic changes to a flow's features can silence an alert altogether, which is known as evasion. The same kind of change can also swap the alert's attack label, which is misattribution. Worse still, the reject class can be exploited to make alerts quietly disappear. Several defences help. The random-forest gate holds up better under this kind of pressure. Sudden shifts in alert volume or in the mix of labels are worth watching for. And the family label should never be the only thing that triggers an automated response.

Poisoning is the second danger. Assume that data can be introduced into the retraining set by an attacker. The model might then be trained to ignore its own traffic. Training data must only originate from reliable sources with restricted access in order to avoid this, and it must be verified prior to each retraining.

The third risk concerns the model itself. In the experiments, white-box attacks were far more damaging than attacks borrowed from another model. So, the trained models, their weights and the reference set all need to be locked away behind access controls. An attacker who cannot see inside the model loses most of that advantage.

Last comes the privacy of the flow data. Flow records show who talked to whom, and when. The system should only be retrained for as long as necessary once they are implemented. Data should be encrypted and safeguarded by access controls and appropriate security measurements.

5.6 Design Constraints and Component Constraints

Design constraints. Splitting the system into two stages comes with a built-in catch. If the gate misses an attack, Stage B never sees it, so that attack can never be given a family label. In effect, attribution can only ever be as good as detection. That is why the robustness analysis concentrated on the gate.

The choice of tree ensembles over deep learning was a trade. Trees train quickly, their

feature importance is easier to interpret, and they cope well with heavily skewed features. The price was giving up the chance to learn straight from raw packet sequences.

The combiner was kept simple on purpose. Its per-class weight matrix overfits less than a stacked meta-classifier would. Then again, it cannot capture the more subtle interactions that a flexible combiner might pick up.

The reject class brings a trade-off of its own, at the boundary between Fuzzers and benign traffic. Fewer false alarms means more attacks slipping through, and the reverse is also true. None of the designs tested here made that tension go away. Size mattered too. Every extra ensemble member adds to the cost of making a prediction. So when a 12-member pool showed no established gain, the team stayed at seven members.

Component constraints. The system is built on CICFlowMeter-style flow features, and it inherits whatever that extractor cannot do. It only ever sees flow statistics, never the payload inside the packets. Its behaviour also shifts with the extractor's flow-timeout and segmentation settings, as the robustness study revealed.

The software libraries add a smaller wrinkle. scikit-learn, XGBoost and imbalanced-learn are all mature tools, but they keep changing. New library versions, together with tree building spread across several threads, cause tiny differences between runs, about ±0.001 macro-F1. For that reason, any two configurations being compared were run inside the same experiment.

Hardware was the final factor. The development machine had Nvidia RTX 3070 GPU. That made no difference for tree-models, though it would have held back any deep-learning alternative.

5.7 Data Constraints and Computational Constraints

Data constraints. All results come from traffic generated in a single testbed. CIC-UNSW-NB15 and UNSW-NB15 share the same underlying traffic, CIC-IDS2017 and TON_IoT proved too easy to discriminate between methods, and none of them provides a chronological test of generalization to future traffic. The class imbalance is extreme: Analysis and Worms have only 77 and 49 test flows in the full release, so their per-class scores have wide confidence intervals (for example, Worms ±0.13 on the curated data). Nearly a third of the curated records were duplicates, and the curated file lacked port and protocol information, which had to be recovered from the full release by feature matching

(94.1% unambiguous). Testbed artifacts such as source addresses and TTL values had to be identified and excluded. No packet captures were available, which ruled out payload-based features, the direction this work identifies as most promising for further gains.

Computational constraints. All work ran on a single CPU workstation. Five-fold cross-fitting of eight models was the dominant cost, about 7–8 minutes on the curated file and longer on the full release, which limited how many configurations could be explored in each experiment (for example, the hyperparameter search covered 40 configurations). The full release had to be loaded in 32-bit precision to fit in memory, and nearest-neighbor computations over large reference sets were the most memory-intensive operations. The bootstrap procedure (2,000 resamples) was cheap by comparison. Even though these constraints did not limit any planned experiments but they influenced the implementation caching and reuse..

5.8 Budget Constraints

This study has effectively no cost. All datasets are freely available for research, all software is open source, and all experiments ran on personally owned hardware, so no cloud computing, license or data-acquisition costs arose. The only indirect costs were electricity for computation and researcher time. The absence of a budget did constrain the project's scope: it excluded commercial IDS products for comparison, paid cloud GPUs that deep-learning baselines would have required, and the collection of new real-world traffic, which would need network equipment, storage and legal and ethical approval. For an organization deploying the system, costs would be limited to a server for model inference, the integration effort with existing flow collectors and security tools, and analyst time for reviewing alerts.

5.9 Task Breakdown and Project Timeline

The project followed an iterative, experiment-driven development cycle within a phase-wise plan. Each cycle introduced one change, evaluated it against held-out data with a paired test, and either kept or discarded it, and the outcome of each cycle determined the next. The work was divided into the phases and tasks shown in Table 5.2.

Table 5.2 Task breakdown

Phase	Task	Purpose
1. Planning and research	Topic selection and literature review	Identify gaps in ML-based NIDS research

Phase	Task	Purpose
	Dataset study and exploratory analysis	Discover imbalance, duplication and feature properties
2. Design and baseline	Leakage-safe preprocessing pipeline	Guarantee valid evaluation
	Baseline models (DT, RF, XGBoost)	Establish reference performance
3. Core development	Imbalance handling and two-stage design	Address rare-family detection
	Ensemble and combiner experiments	Combine imbalance-diverse members
4. Information and realism	Feature engineering and service identity	Test whether richer information helps
	Full-release evaluation and reject class	Evaluate at the realistic base rate
5. Validation	Significance testing and ablations	Establish which components matter
	Robustness experiments	Measure reliability under perturbation
6. Documentation	Report writing, review and revision	Communicate and defend the work

5.10 Milestones and Deliverables

The project was organised around six milestones, each with a measurable completion criterion and a concrete deliverable (Table 5.3).

Table 5.3 Milestones and deliverables

Milestone	Completion criterion	Deliverable
M1: Problem defined	Literature reviewed; dataset audited	Literature extraction workbook; dataset audit notes
M2: Baselines complete	Leakage-safe pipeline and three baselines evaluated	Baseline notebooks with per-class reports
M3: Proposed model complete	Two-stage ensemble outperforms baselines with established significance	Two-stage and ensemble notebooks; curated-dataset results
M4: Final model validated	Full-release system, reject class and ablations complete	Final model, significance study, result logs
M5: Robustness evaluated	Noise, evasion and misattribution experiments complete	Robustness scripts and results
M6: Thesis submitted	Report written, reviewed and revised	Thesis report and presentation

5.11 Gantt Chart

The project schedule is shown as a Gantt chart in Figure 5.1. Each horizontal bar represents a task from Section 5.9. A bar's length indicates the duration of the assignment, and its

beginning indicates the week it began. Color tells you which phase a task belongs to. Grey covers planning and research. Blue marks design and core development, while green is used for information and validation. Robustness work appears in orange, and documentation in red. The six milestones, labelled M1 to M6, are shown as dotted vertical lines and match those described in Section 5.10. Where bars overlap, tasks were running at the same time. Report writing is a good example. It started while the later experiments were still under way. Reading the chart from top to bottom also shows the dependencies, because each core task built on what the one above it had produced.

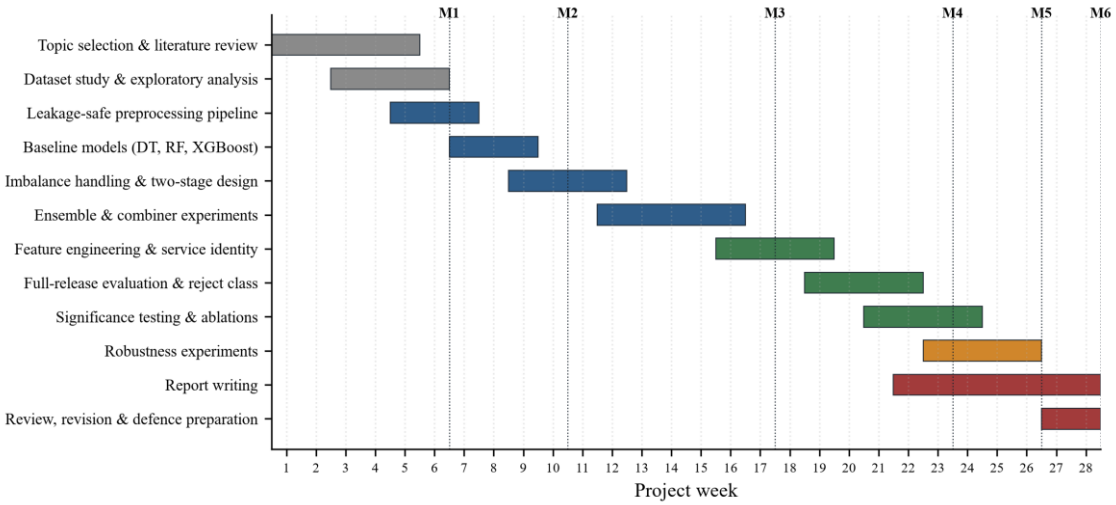


Figure 5.1 Project Gantt chart (indicative weeks)

5.12 Summary

This chapter stepped back to look at the system in a professional setting. It lines up with the main standards for intrusion detection, cybersecurity and AI risk. Since it is still a research prototype, though, no formal audit has been carried out. The tree-based design is efficient and scales well. Its benefits to society are clear. So are the risks, chiefly misuse and leaning on it too heavily.

Ethical concerns were dealt with in several ways. The data came from synthetic testbeds rather than real users. Results were averaged across classes so that rare attacks counted equally. Features that offered misleading shortcuts were left out, and the findings were reported honestly.

The project also worked within real limits. All the data came from a single testbed, and the classes were extremely imbalanced. The features described flows only, with no payload, and every experiment ran on a CPU. Each of these was handled with care. A careful pipeline

design, statistical testing and caching helped, along with clear statements of what the work does and does not cover. In the end, the project was completed in six phases and six milestones, with no direct financial cost..

Chapter 6

Conclusion

6.1 Summary of the Study / Work Done

This thesis set out to build a machine-learning network intrusion detection system that not only detects attacks but also identifies their family, including the rare families that standard models miss. It also set out to evaluate that system without the data leakage and accuracy-centered reporting common in the literature. Using the CIC-UNSW-NB15 dataset, a leakage-safe pipeline was built that removes duplicates before splitting, fits every label-dependent step on training data only, fits ensemble weights on cross-fitted predictions and scores the test set once. The work began with a set of flat baseline models, which reached a macro-F1 between 0.534 and 0.581. From there, a series of controlled trials, each one tested statistically, gradually shaped a two-stage system. A detection gate sits at the front. Behind it is an ensemble that works out which family an attack belongs to. Its members differ in how strongly they rebalance the classes, and a learnt per-class weight matrix combines their votes. Performance then rose further thanks to tempered class weighting, engineered behavioural features and recovered service identity. They lifted the system to a macro-F1 of 0.746 [0.727, 0.763] on the curated dataset. After adding a reject class and an end-to-end combiner target, it obtained macro-F1 0.687 [0.671, 0.704] at 98.7% accuracy at the realistic 97.5% benign base rate of the entire release Error analysis attributed the remaining mistakes to class overlap in the flow representation, and a robustness study showed how detection and attribution degrade under perturbation.

The research questions were answered as follows. The first research question concerned class imbalance. Moderation won. Setting α to 0.25 worked best. It beat both no rebalancing and full rebalancing. In fact, it was the biggest modelling lever of all, adding 0.039.

The second question tested the two-stage design. It helped only alongside tempered weighting, adding 0.018. The combination mattered, not the hierarchy alone.

The third question asked what limits performance. Information mattered far more than model capacity. Service identity added 0.098, while making the model bigger brought no lasting gain.

The fourth question looked at realistic traffic. The system stayed accurate overall, and the

reject class cut benign false alarms fourfold. Most of the remaining false alarms sat along the Fuzzers boundary.

Robustness was the subject of the fifth question. Modest changes in input could cause the system to malfunction. Although a targeted attacker may still get around it or tamper with the family labels, a random-forest gate maintained detection under noise at a low clean cost.

6.2 Key Contributions

The main contributions of this research are:

- **A leakage-safe, imbalance-aware two-stage ensemble** for multi-class flow-based intrusion detection, which improved macro-F1 by +0.050 over a flat XGBoost baseline through modelling alone ($p < 0.001$) and reached 0.687 at the realistic base rate.
- **The principle of moderate rebalancing**, supported by two independent weighting families and by the failure of full SMOTE and "balanced" weighting, together with an imbalance-diverse ensemble design in which members are chosen for complementarity rather than individual accuracy.
- **Evidence that information outweighs model capacity on this problem.** Service identity gave the largest gain in the project (+0.098), while hyperparameter tuning, dynamic selection, threshold tuning and wider pools were all not established. The class-overlap analysis explains the plateau.
- **A reject class and an end-to-end combiner objective** that together add +0.027 macro-F1 and reduce benign false alarms fourfold at the realistic base rate.
- **A physically consistent robustness evaluation**, showing that unconstrained perturbation overstates damage, that the gate model is the main robustness lever, and that misattribution is a stealthy attacker objective invisible to evasion-only evaluation.
- **A transparent evaluation protocol**, with cross-fitting, a single test-set evaluation, paired bootstrap tests and logged runs, and a complete record of negative results.

6.3 Limitations of the Study

The study has several limitations. All results come from traffic generated in a single testbed, so generalization to other networks has not been demonstrated: CIC-IDS2017 and

TON_IoT proved too easy to serve as meaningful second tests, and no chronological split was possible. Part of the service-identity gain may reflect how the testbed's traffic generator targeted services rather than how real attacks behave. Two attack families, Analysis and Worms, have so few test flows that their scores carry wide confidence intervals, and Analysis remains poorly detected (F1 0.14). The system uses only flow statistics, with no packet payload, which this work identifies as the binding constraint on further improvement. The confidence intervals capture test-set sampling variation but not variation from different random splits or seeds, so they are somewhat optimistic. Small changes to the input can throw the model off, and a determined attacker can manipulate it. Explainability was also left out. No feature-attribution method, such as SHAP, was applied. Lastly, every test ran offline on recorded data. The system was never deployed in a live network or checked for real-time speed.

6.4 Future Work

Several directions follow from these findings. First, since information rather than model capacity limited performance, future work should enrich the representation with payload-derived or session-level features, which requires packet captures rather than flow records. Second, cross-network and chronological evaluation, for example training on one dataset and testing on another, or using timestamps to train on earlier traffic and test on later traffic, would show whether the approach generalizes beyond a single testbed. Explainability is the next gap to fill. Tools such as SHAP (Lundberg & Lee, 2017) could help, used as a diagnostic rather than a general overview. They would be most useful aimed at the family pairs the system mixes up most, namely Exploits and Reconnaissance, DoS and Exploits, and Fuzzers and Shellcode. The robustness results point somewhere too. It makes sense to rebuild the pipeline around a random-forest gate. Adversarial training and monitoring for misattribution are also worth exploring, along with better protection for the reject class. Real-world use is another open question. A deployment study could measure how quickly each flow is processed and how much traffic the system can handle. It could also connect the system to a live flow collector and see how much work it creates for analysts in a security operations center. And finally, there is the problem of scarce labels. Rare families have the fewest labelled examples, and semi-supervised or active learning could ease the system’s dependence on them.

References

Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Transactions on Emerging Telecommunications Technologies, 32*(1), Article e4150. https://doi.org/10.1002/ett.4150

Ahmed, U., Jiangbin, Z., Almogren, A., Khan, S., Sadiq, M. T., Altameem, A., & Rehman, A. U. (2024). Explainable AI-based innovative hybrid ensemble model for intrusion detection. *Journal of Cloud Computing, 13*(1), Article 150. https://doi.org/10.1186/s13677-024-00712-x

Almuhanna, R., & Dardouri, S. (2025). A deep learning/machine learning approach for anomaly based network intrusion detection. *Frontiers in Artificial Intelligence, 8*, Article 1625891. https://doi.org/10.3389/frai.2025.1625891

Apruzzese, G., Andreolini, M., Ferretti, L., Marchetti, M., & Colajanni, M. (2022). Modeling realistic adversarial attacks against network intrusion detection systems. *Digital Threats: Research and Practice, 3*(3), Article 31. https://doi.org/10.1145/3469659

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *Proceedings of the 31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association.

Ben Ncir, C. E., Ben HajKacem, M. A., & Alattas, M. (2024). Enhancing intrusion detection performance using explainable ensemble deep learning. *PeerJ Computer Science, 10*, Article e2289. https://doi.org/10.7717/peerj-cs.2289

Bertoli, G. de C., Pereira Júnior, L. A., Saotome, O., dos Santos, A. L., Verri, F. A. N., Marcondes, C. A. C., Barbieri, S., Rodrigues, M. S., & de Oliveira, J. M. P. (2021). An end-to-end framework for machine learning-based network intrusion detection system. *IEEE Access, 9*, 106790–106805. https://doi.org/10.1109/ACCESS.2021.3101188

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the*

22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Cruz, R. M. O., Sabourin, R., & Cavalcanti, G. D. C. (2018). Dynamic classifier selection: Recent advances and perspectives. *Information Fusion, 41*, 195–216. https://doi.org/10.1016/j.inffus.2017.09.010

Cui, Y., Jia, M., Lin, T.-Y., Song, Y., & Belongie, S. (2019). Class-balanced loss based on effective number of samples. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 9260–9269). IEEE. https://doi.org/10.1109/CVPR.2019.00949

Dasari, A. K., Biswas, S. K., & Purkayastha, B. (2026). ASENIDSB: A stacked ensemble-based network intrusion detection system optimized with Boruta feature selection. *Journal of Intelligent & Fuzzy Systems, 50*(4), 1073–1092. https://doi.org/10.1177/18758967251358591

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall/CRC.

Emirmahmutoğlu, E., & Atay, Y. (2025). A feature selection-driven machine learning framework for anomaly-based intrusion detection systems. *Peer-to-Peer Networking and Applications, 18*(3), Article 161. https://doi.org/10.1007/s12083-025-01947-4

Farhan, M., Waheed ud din, H., Ullah, S., Hussain, M. S., Khan, M. A., Mazhar, T., Khattak, U. F., & Jaghdam, I. H. (2025). Network-based intrusion detection using deep learning technique. *Scientific Reports, 15*, Article 25550. https://doi.org/10.1038/s41598-025-08770-0

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning, 63*(1), 3–42. https://doi.org/10.1007/s10994-006-6226-1

Injadat, M., Moubayed, A., Nassif, A. B., & Shami, A. (2021). Multi-stage optimized machine learning framework for network intrusion detection. *IEEE Transactions on Network and Service Management, 18*(2), 1803–1816. https://doi.org/10.1109/TNSM.2020.3014929

International Organization for Standardization. (2022). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (ISO/IEC Standard No. 27001:2022).

International Organization for Standardization. (2023a). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023).

International Organization for Standardization. (2023b). *Information technology — Artificial intelligence — Guidance on risk management* (ISO/IEC Standard No. 23894:2023).

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation,

detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), Article 15. https://doi.org/10.1145/2382577.2382579

Khan, M. A. (2021). HCRNNIDS: Hybrid convolutional recurrent neural network-based network intrusion detection system. *Processes, 9*(5), Article 834. https://doi.org/10.3390/pr9050834

Ko, A. H. R., Sabourin, R., & Britto, A. S., Jr. (2008). From dynamic classifier selection to dynamic ensemble selection. *Pattern Recognition, 41*(5), 1718–1731. https://doi.org/10.1016/j.patcog.2007.10.015

Lashkari, A. H., Draper-Gil, G., Mamun, M. S. I., & Ghorbani, A. A. (2017). Characterization of Tor traffic using time based features. In *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP 2017)* (pp. 253–262). SciTePress. https://doi.org/10.5220/0006105602530262

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research, 18*(17), 1–5.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *2017 IEEE International Conference on Computer Vision (ICCV)* (pp. 2999–3007). IEEE. https://doi.org/10.1109/ICCV.2017.324

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.

Mohammadian, H., Lashkari, A. H., & Ghorbani, A. A. (2024). Poisoning and evasion: Deep learning-based NIDS under adversarial attacks. In *2024 21st Annual International Conference on Privacy, Security and Trust (PST)* (pp. 1–9). IEEE. https://doi.org/10.1109/PST62714.2024.10788064

Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society, 72*, Article 102994. https://doi.org/10.1016/j.scs.2021.102994

Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)* (pp. 1–6). IEEE. https://doi.org/10.1109/MilCIS.2015.7348942

Nassreddine, G., Nassereddine, M., & Al-Khatib, O. (2025). Ensemble learning for network intrusion detection based on correlation and embedded feature selection techniques. *Computers, 14*(3),

Article 82. https://doi.org/10.3390/computers14030082

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

National Institute of Standards and Technology. (2024). *The NIST cybersecurity framework (CSF) 2.0* (NIST CSWP 29). U.S. Department of Commerce. https://doi.org/10.6028/NIST.CSWP.29

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Qazi, E. U. H., Faheem, M. H., & Zia, T. (2023). HDLNIDS: Hybrid deep-learning-based network intrusion detection system. *Applied Sciences, 13*(8), Article 4921. https://doi.org/10.3390/app13084921

Scarfone, K., & Mell, P. (2007). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication 800-94). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-94

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)* (pp. 108–116). SciTePress. https://doi.org/10.5220/0006639801080116

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy* (pp. 305–316). IEEE. https://doi.org/10.1109/SP.2010.25

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645–3650). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1355

Tafreshian, B., & Zhang, S. (2024). A defensive framework against adversarial attacks on machine learning-based network intrusion detection systems. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)* (pp. 2436–2441). IEEE. https://doi.org/10.1109/TrustCom63139.2024.00337

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP

99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (pp. 1–6). IEEE. https://doi.org/10.1109/CISDA.2009.5356528

Thockchom, N., Singh, M. M., & Nandi, U. (2023). A novel ensemble learning-based model for network intrusion detection. *Complex & Intelligent Systems, 9*(5), 5693–5714. https://doi.org/10.1007/s40747-023-01013-7

Uddamari, N., & Sammulal, P. (2025). CyberAdaptAI: A dynamic ensemble learning framework for real-time cyberattack detection using AdaptEnsembleNet. *SSRG International Journal of Electronics and Communication Engineering, 12*(9), 11–31. https://doi.org/10.14445/23488549/IJECE-V12I9P102

Uppal, M., Gulzar, Y., Gupta, D., Uppal, J., Kumar, M., & Saini, S. (2025). Enhancing accuracy through ensemble based machine learning for intrusion detection and privacy preservation over the network of smart cities. *Discover Internet of Things, 5*, Article 11. https://doi.org/10.1007/s43926-025-00101-z

Vitorino, J., Praça, I., & Maia, E. (2023). SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection. *Computers & Security, 134*, Article 103433. https://doi.org/10.1016/j.cose.2023.103433

Wolpert, D. H. (1992). Stacked generalization. *Neural Networks, 5*(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

Woods, K., Kegelmeyer, W. P., & Bowyer, K. (1997). Combination of multiple classifiers using local accuracy estimates. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 19*(4), 405–410. https://doi.org/10.1109/34.588027

Appendix

Appendix A: Engineered Behavioural Features

All features are computed per flow as $x / (y + 10^{-6})$, with any infinite or undefined value set to 0. "Fwd" and "Bwd" denote the forward and backward directions. P = packets, B = bytes.

Table A.1 Formulas of the 31 engineered behavioural features

Feature	Formula
r_pkt_fwd_bwd	Fwd P / Bwd P
r_byte_fwd_bwd	Fwd B / Bwd B
asym_pkt	(Fwd P − Bwd P) / (Fwd P + Bwd P)
asym_byte	(Fwd B − Bwd B) / (Fwd B + Bwd B)
bpp_fwd	Fwd B / Fwd P
bpp_bwd	Bwd B / Bwd P
bpp_flow	(Fwd B + Bwd B) / (Fwd P + Bwd P)
r_bpp	bpp_fwd / bpp_bwd
cv_flow_iat	Flow IAT Std / Flow IAT Mean
cv_fwd_iat	Fwd IAT Std / Fwd IAT Mean
cv_bwd_iat	Bwd IAT Std / Bwd IAT Mean
iat_span_rel	(Flow IAT Max − Flow IAT Min) / Flow IAT Mean
cv_fwd_len	Fwd Packet Length Std / Fwd Packet Length Mean
cv_bwd_len	Bwd Packet Length Std / Bwd Packet Length Mean
cv_pkt_len	Packet Length Std / Packet Length Mean
fwd_len_span_rel	(Fwd Packet Length Max − Min) / Fwd Packet Length Mean
bwd_len_span_rel	(Bwd Packet Length Max − Min) / Bwd Packet Length Mean
frac_FIN, frac_SYN, frac_RST, frac_PSH, frac_ACK	Flag count / (Fwd P + Bwd P)
r_syn_fin	SYN Flag Count / FIN Flag Count
hdr_per_pkt_fwd	Fwd Header Length / Fwd P
hdr_per_pkt_bwd	Bwd Header Length / Bwd P
payload_share_fwd	(Fwd B − Fwd Header Length) / Fwd B
active_share	Active Mean / Flow Duration
idle_share	Idle Mean / Flow Duration
r_active_idle	Active Mean / Idle Mean
dur_per_pkt	Flow Duration / (Fwd P + Bwd P)
r_init_win	FWD Init Win Bytes / Bwd Init Win Bytes

Appendix B: Reproducibility Map

Every result in Chapter 4 can be regenerated from the project repository. Table A.2 maps

the main results to the notebook or script that produced them. Scripts in the results directory are stored with their log (.txt) and results (.json) files.

Table A.2 Source of each main result

Result	Source
Flat baselines (Table 4.2)	decision_tree.ipynb, random_forest.ipynb, xgboost_model.ipynb (and _mi_rfecv variants)
Imbalance-handling study	two_stage_stageB_weighting.ipynb, two_stage_stageB_experiments.ipynb
Two-stage architecture and ensembles	two_stage_hierarchical.ipynb, two_stage_stageB_ensemble_v2.ipynb, two_stage_stageB_ensemble_v3.ipynb
Engineered features and v6	feature_engineering.ipynb, final_model_v6.ipynb, v6_significance.ipynb
Service identity	results/cic_unsw_svc/
Full release, reject class, end-to-end objective	final_model_v7_full.ipynb, final_model_v7_1_reject.ipynb, results/v71_w_e2e/
Significance study	ablation_significance.ipynb
Negative results	two_stage_stageB_des.ipynb, two_stage_thresholds.ipynb, two_stage_gate_weighting.ipynb, feature_engineering_v2.ipynb, results/v72_hparam/, results/v71c_reject_enrich/, results/v71d_region_w/, results/v71e_view_pool/
Error analysis	label_ambiguity_ceiling.ipynb, knn_class_overlap.ipynb
Robustness	results/v73_adv_budget/ to results/v73g_controllability_aware/
Other datasets	final_model_v6_unsw.ipynb, results/cicids2017_v6/, results/toniot_audit/, results/unsw_dedupe_ablation/
Figures in this report	thesis/figures/make_report_figures.py, thesis/figures/make_fig_protocol.py